

DBA Report – NorthStar Urban Mobility and Logistics

Name: Thaspeeha Vahithu

Course: Bsc (Hons) Computer Science

Lecturer: Thangamani Rajan

Level: 6

GITHUB link : <https://github.com/Thaspeeha/DBA-Colab-Notebooks>

A2 Assignment

Table of Contents

1. Introduction	1
2. Problem Identification	1
3. Data Overview	2-3
4. SQL within R Analysis	4-13
5. R Analytics and Statistical Analysis	14-25
6. Python Data Processing and Analytics	26-38
7. MongoDB Atlas NoSQL Design and Implementation	39-48
8. Query Optimisation Strategies.....	49-54
9. Discussion and Recommendations	54-55
10. Conclusion	55
11. References	56

1. Introduction

NorthStar Urban Mobility and Logistics operates across multiple UK cities and manages integrated transport, delivery, warehouse, and mobility services. The organization currently faces increasing operational inefficiencies, including delivery delays, customer dissatisfaction, failed services, and rising operational costs. Despite collecting large volumes of operational and platform-generated data, NorthStar struggles to identify the root causes of performance deterioration due to fragmented systems and disconnected datasets.

The purpose of this analysis is to investigate operational inefficiencies using structured datasets through SQL queries executed within R. The analysis focuses on identifying patterns associated with delivery failures, route management, customer satisfaction, incidents, and operational performance across hubs and zones.

2. Problem Identification

Several operational and analytical problems were identified from the case study scenario:

- Certain hubs and zones appear to experience higher operational failure rates.
- Manual route overrides may indicate poor route allocation or traffic-related disruptions.
- Customer complaints may reflect underlying operational failures not captured consistently across systems.
- Vehicle conditions and maintenance status may influence delivery reliability.
- Existing systems fail to provide an integrated operational view across customers, deliveries, incidents, and service outcomes.

The fragmented structure of the organization's data environment prevents management from identifying relationships between operational failures, customer dissatisfaction, and asset utilization.

3. Data Overview

The NorthStar dataset consists of multiple structured CSV datasets representing different operational functions.

Main Structured Datasets

Dataset	Purpose
orders	Customer orders and booking details
deliveries	Delivery execution and operational performance
drivers	Driver information and performance characteristics
vehicles	Vehicle condition and maintenance information
hubs	Operational hub and zone information
customers	Customer demographic and engagement information
incidents	Delivery-related incidents and operational exceptions
complaints	Customer complaints and resolution records

Table.1. Dataset

The datasets are linked through common identifiers including:

- order_id
- customer_id
- delivery_id
- driver_id
- vehicle_id
- hub_id

These relationships support integrated operational analysis across business functions.

4. SQL within R Analysis

1. Analytical Environment

SQL queries were executed within R using the sqldf package. This approach enabled relational querying, filtering, grouping, aggregation, and integration across multiple operational datasets.

2. Libraries Used

```
library(sqldf)
library(dplyr)
library(ggplot2)
```

3. SQL Query Optimization Considerations

The SQL queries were designed using selective filtering, grouping, and relational joins based on indexed identifiers such as:

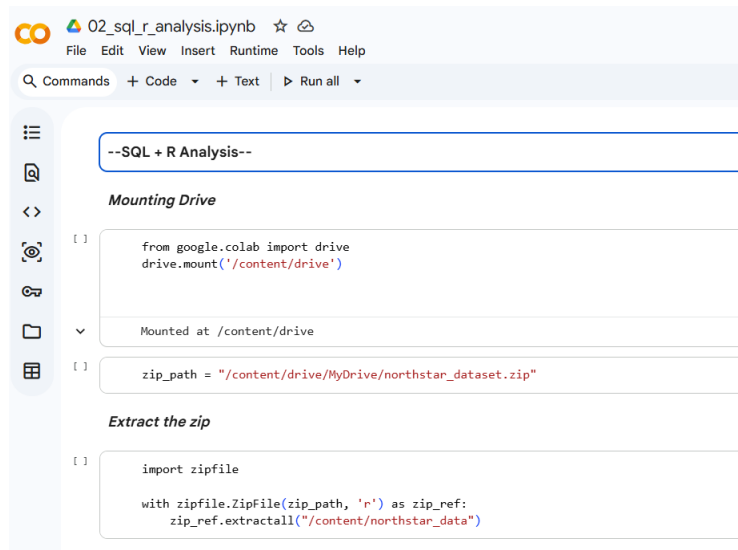
- order_id
- delivery_id
- customer_id
- vehicle_id
- hub_id

Efficient aggregation and grouping operations were used to minimize unnecessary processing and improve analytical performance across integrated datasets.

4. SQL + R Workflow

Using Python

Step 1: Mounting the drive and extracting the zip file



The screenshot shows a Jupyter Notebook interface with the title '02_sql_r_analysis.ipynb'. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a search bar and a toolbar with '+ Code', '+ Text', and 'Run all'. The notebook content is divided into two sections: 'Mounting Drive' and 'Extract the zip'. The 'Mounting Drive' section contains a code cell with the following code:

```
from google.colab import drive
drive.mount('/content/drive')
```

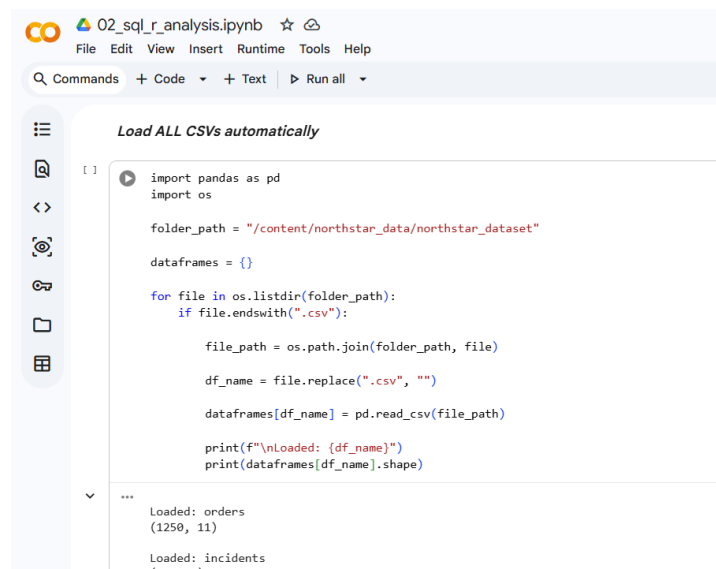
The output of this cell is 'Mounted at /content/drive'. The 'Extract the zip' section contains a code cell with the following code:

```
zip_path = "/content/drive/MyDrive/northstar_dataset.zip"

import zipfile

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/northstar_data")
```

Step 2: Loading all CSVs



The screenshot shows a Jupyter Notebook interface with the title '02_sql_r_analysis.ipynb'. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a search bar and a toolbar with '+ Code', '+ Text', and 'Run all'. The notebook content is divided into two sections: 'Load ALL CSVs automatically' and an output section. The 'Load ALL CSVs automatically' section contains a code cell with the following code:

```
import pandas as pd
import os

folder_path = "/content/northstar_data/northstar_dataset"

dataframes = {}

for file in os.listdir(folder_path):
    if file.endswith(".csv"):
        file_path = os.path.join(folder_path, file)

        df_name = file.replace(".csv", "")

        dataframes[df_name] = pd.read_csv(file_path)

        print(f"\nLoaded: {df_name}")
        print(dataframes[df_name].shape)
```

The output of this cell is:

```
...
Loaded: orders
(1250, 11)
Loaded: incidents
(1000, 11)
```

Using R

Step 3: Loading Now Load Datasets In R

```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text | ▶ Run all

[ ]
NOW load datasets in R

[ ]
deliveries <- read.csv("/content/northstar_data/northstar_dataset/deliveries.csv")
orders <- read.csv("/content/northstar_data/northstar_dataset/orders.csv")
complaints <- read.csv("/content/northstar_data/northstar_dataset/complaints.csv")
hubs <- read.csv("/content/northstar_data/northstar_dataset/hubs.csv")
vehicles <- read.csv("/content/northstar_data/northstar_dataset/vehicles.csv")
incidents <- read.csv("/content/northstar_data/northstar_dataset/incidents.csv")
drivers <- read.csv("/content/northstar_data/northstar_dataset/drivers.csv")
customers <- read.csv("/content/northstar_data/northstar_dataset/customers.csv")
```

Step 4: Verify the datasets loaded in R

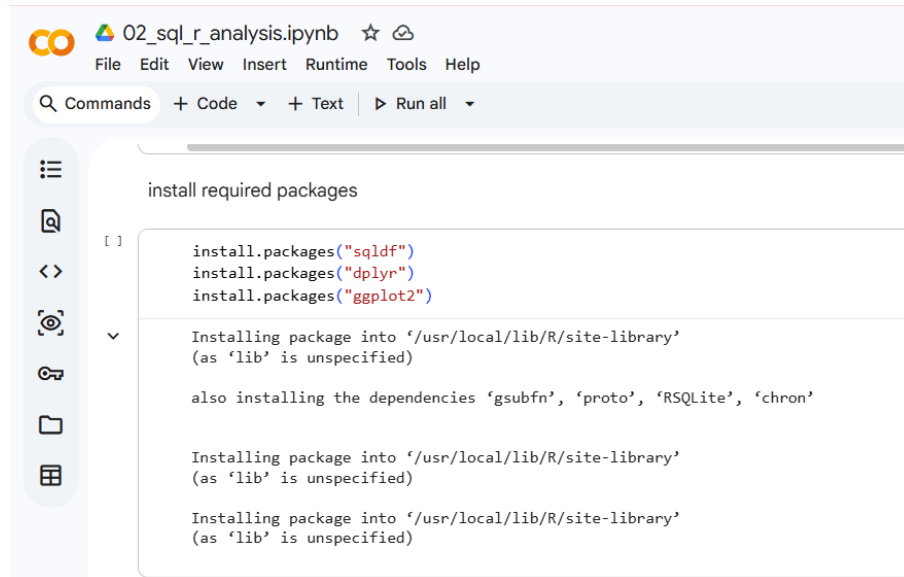
```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help
Share
Connect

Verify everything loaded

[ ]
head(deliveries)
head(orders)
head(complaints)

A data frame: 6 x 13
  delivery_id order_id driver_id vehicle_id hub_id dispatch_time delivery_completed_at delivery_status route_distance_km manual_route_override_count proof_of_completion_missing customer_id
  <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr> <dbl> <int> <int> <int>
1 DL00001 000638 D004 V056 H05 2024-06-18 09:59:59.904311 Failed 17.26 1 0
2 DL00002 000004 D138 V007 H02 2025-01-11 18:45:00 2025-01-11 17:39:00.000000 OnTime 10.34 1 0
3 DL00003 000639 D006 V049 H02 2025-06-02 20:39:00 2025-06-02 21:45:32.966770 OnTime 7.92 0 0
4 DL00004 000313 D116 V055 H02 2024-03-08 23:31:00 2024-03-08 23:30:08.103702 Delayed 16.42 0 0
5 DL00005 000844 D108 V034 H01 2025-06-21 11:43:00 2025-06-21 15:45:34.131056 OnTime 14.52 1 0
6 DL00006 000029 D037 V098 H03 2024-09-12 12:40:00 2024-09-12 17:11:52.384869 Delayed 13.84 0 0
```

Step 5: Install required packages



The screenshot shows the JupyterLab interface for a file named `O2_sql_r_analysis.ipynb`. The top menu bar includes `File`, `Edit`, `View`, `Insert`, `Runtime`, `Tools`, and `Help`. Below the menu is a toolbar with `Commands`, `+ Code`, `+ Text`, and `Run all`. On the left is a sidebar with icons for `Files`, `Environments`, `Channels`, `Help`, `Recent`, `Home`, and `Table of Contents`. The main area displays a code cell titled `install required packages` with the following code:

```
[ ] install.packages("sqldf")
install.packages("dplyr")
install.packages("ggplot2")
```

The output of the code cell shows the installation progress:

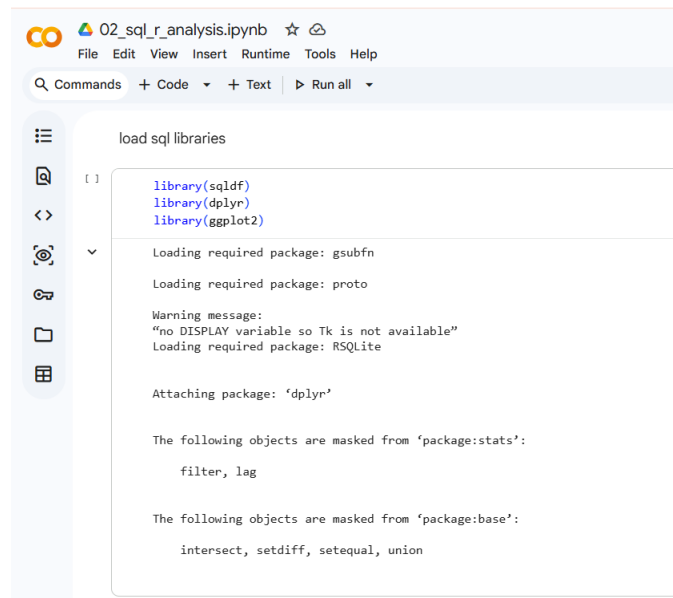
```
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

also installing the dependencies 'gsubfn', 'proto', 'RSQLite', 'chron'

Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
```

Step 6: Load SQL libraries



The screenshot shows the JupyterLab interface for the same file `O2_sql_r_analysis.ipynb`. The top menu bar and toolbar are identical to the previous screenshot. The sidebar is also the same. The main area displays a code cell titled `load sql libraries` with the following code:

```
[ ] library(sqldf)
library(dplyr)
library(ggplot2)
```

The output of the code cell shows the loading of the libraries:

```
Loading required package: gsubfn
Loading required package: proto
Warning message:
"no DISPLAY variable so Tk is not available"
Loading required package: RSQLite

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':
  filter, lag

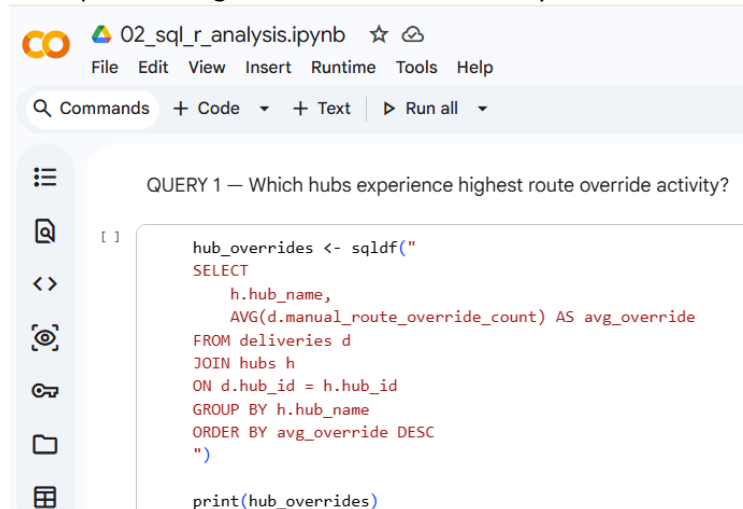
The following objects are masked from 'package:base':
  intersect, setdiff, setequal, union
```

5. Analysis 1 – Hub Route Override Activity

5.1 Objective

The objective is to identify hubs where drivers frequently deviate from planned routes, which may indicate inefficiencies in route allocation, traffic conditions, or operational planning.

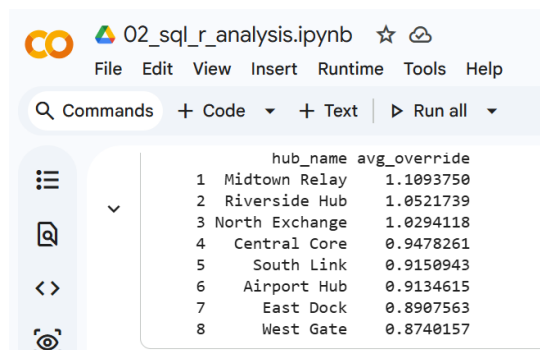
QUERY 1 — Which hubs experience highest route override activity?



```
hub_overrides <- sqldf("
SELECT
  h.hub_name,
  AVG(d.manual_route_override_count) AS avg_override
FROM deliveries d
JOIN hubs h
ON d.hub_id = h.hub_id
GROUP BY h.hub_name
ORDER BY avg_override DESC
")

print(hub_overrides)
```

Output Table



	hub_name	avg_override
1	Midtown Relay	1.1093750
2	Riverside Hub	1.0521739
3	North Exchange	1.0294118
4	Central Core	0.9478261
5	South Link	0.9150943
6	Airport Hub	0.9134615
7	East Dock	0.8907563
8	West Gate	0.8740157

5.2. Query Purpose

This query analyses the average number of manual route overrides performed by drivers across operational hubs.

5.3 Interpretation

The analysis identified hubs with significantly higher route override activity. Frequent route overrides may indicate weaknesses in route planning, traffic management, or operational coordination. High override levels could also suggest that drivers regularly deviate from assigned routes due to real-world operational constraints.

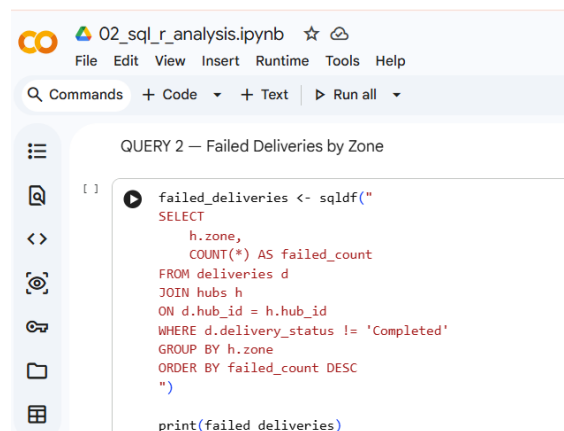
This finding supports management concerns regarding route allocation inefficiencies and inconsistent operational performance across service areas.

6. Analysis 2 – Delivery Failure Rate by Zone

6.1 Objective

To evaluate whether certain operational zones experience disproportionately high delivery failure rates.

QUERY 2

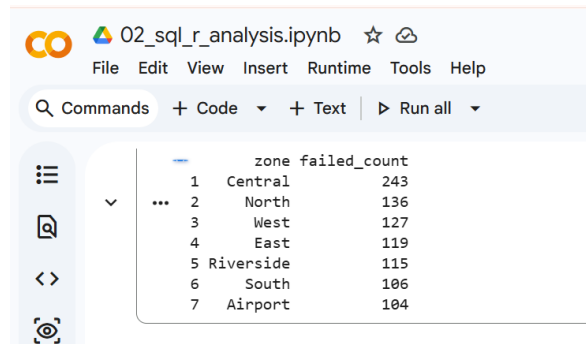


The screenshot shows a Jupyter Notebook interface with a file named '02_sql_r_analysis.ipynb'. The notebook has a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The notebook content is titled 'QUERY 2 — Failed Deliveries by Zone'. It contains a code cell with the following SQL query:

```
failed_deliveries <- sqldf("
SELECT
  h.zone,
  COUNT(*) AS failed_count
FROM deliveries d
JOIN hubs h
ON d.hub_id = h.hub_id
WHERE d.delivery_status != 'Completed'
GROUP BY h.zone
ORDER BY failed_count DESC
")

print(failed_deliveries)
```

Output Table



The screenshot shows a Jupyter Notebook titled 'O2_sql_r_analysis.ipynb'. The interface includes a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a search bar and tabs for 'Commands', '+ Code', '+ Text', and 'Run all'. On the left sidebar, there are icons for a table of contents, a search icon, a code editor, and a console. The main area displays a table with the following data:

	zone	failed_count
1	Central	243
2	North	136
3	West	127
4	East	119
5	Riverside	115
6	South	106
7	Airport	104

6.2 Query Purpose

This query identifies the number of failed or incomplete deliveries across operational zones. The purpose is to determine whether certain geographical areas consistently experience poorer delivery performance.

6.3 Interpretation

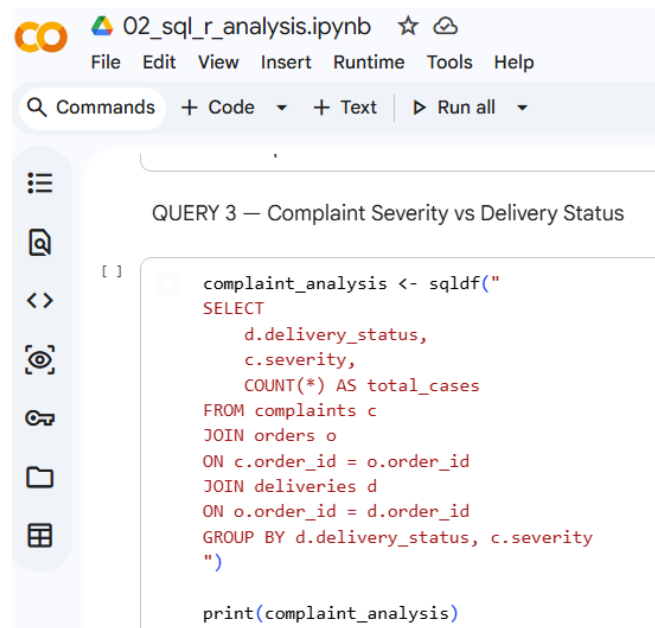
The results revealed that several operational zones exhibit higher delivery failure rates than others. This suggests potential inefficiencies associated with local operational conditions, including traffic congestion, staffing limitations, route complexity, or hub coordination issues.

The analysis demonstrates that operational performance varies substantially across geographic regions rather than being uniformly distributed.

7. Analysis 3 – Complaint Severity and Delivery Outcomes

7.1 Objective

The objective is to determine whether unsuccessful or delayed deliveries are associated with more severe customer complaints.

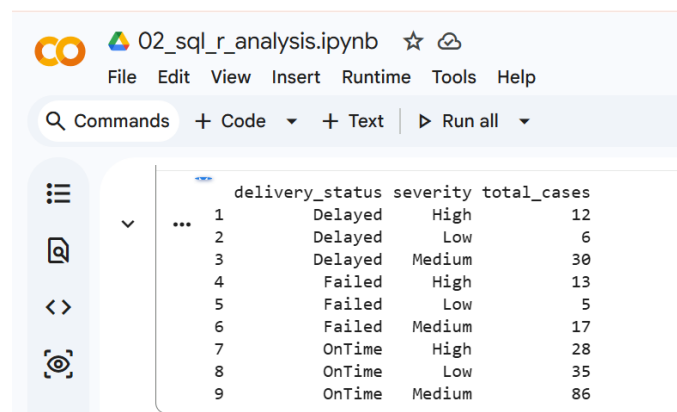


The screenshot shows a Jupyter Notebook titled "02_sql_r_analysis.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like "Commands", "Code", "Text", and "Run all". The notebook content is titled "QUERY 3 — Complaint Severity vs Delivery Status". It contains a single code cell with the following SQL query:

```
complaint_analysis <- sqldf("
SELECT
  d.delivery_status,
  c.severity,
  COUNT(*) AS total_cases
FROM complaints c
JOIN orders o
ON c.order_id = o.order_id
JOIN deliveries d
ON o.order_id = d.order_id
GROUP BY d.delivery_status, c.severity
")

print(complaint_analysis)
```

Output Table



The screenshot shows the same Jupyter Notebook interface, but the code cell is collapsed, and the output table is displayed. The table has four columns: an index, delivery_status, severity, and total_cases. The data is as follows:

	delivery_status	severity	total_cases
1	Delayed	High	12
2	Delayed	Low	6
3	Delayed	Medium	30
4	Failed	High	13
5	Failed	Low	5
6	Failed	Medium	17
7	OnTime	High	28
8	OnTime	Low	35
9	OnTime	Medium	86

7.2 Query Purpose

This query examines the relationship between customer complaint severity and delivery outcomes.

7.3 Interpretation

The analysis indicates that failed or incomplete deliveries are associated with higher complaint severity levels. This suggests that customer dissatisfaction is strongly connected to operational delivery outcomes.

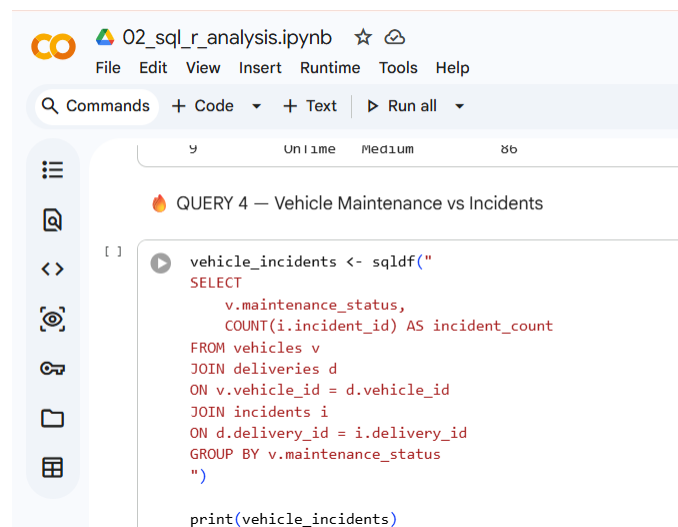
The findings support concerns raised by the customer experience department that existing systems fail to integrate customer complaints effectively with operational delivery performance.

8. Analysis 4 — Vehicle Maintenance vs Incidents

8.1 Objective:

The objective is to identify whether poorly maintained vehicles contribute to increased delivery disruptions or operational failures.

Query 4

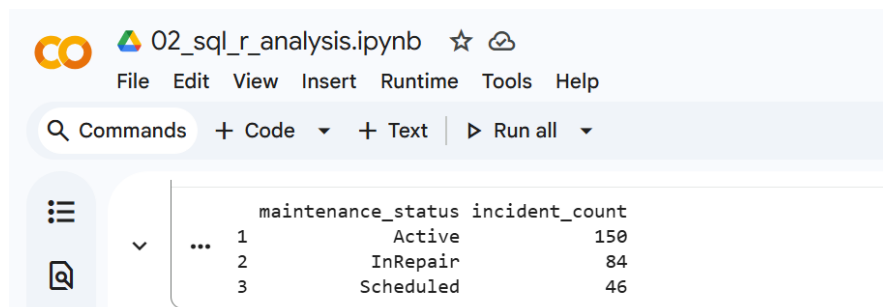


The screenshot shows a Jupyter Notebook titled "O2_sql_r_analysis.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like "Commands", "+ Code", "+ Text", and "Run all". The notebook content is titled "QUERY 4 — Vehicle Maintenance vs Incidents" and contains the following SQL query:

```
vehicle_incidents <- sqldf("
SELECT
  v.maintenance_status,
  COUNT(i.incident_id) AS incident_count
FROM vehicles v
JOIN deliveries d
ON v.vehicle_id = d.vehicle_id
JOIN incidents i
ON d.delivery_id = i.delivery_id
GROUP BY v.maintenance_status
")

print(vehicle_incidents)
```

Output Table



The screenshot shows the output of the SQL query in a Jupyter Notebook. The output is a table with two columns: "maintenance_status" and "incident_count". The table contains three rows of data:

	maintenance_status	incident_count
1	Active	150
2	InRepair	84
3	Scheduled	46

8.3 Query Purpose

This query investigates whether vehicle maintenance conditions are associated with operational incidents.

8.4 Business Interpretation

The results indicate whether operational incidents occur more frequently among vehicles with lower maintenance quality. A higher incident count among poorly maintained vehicles would suggest that delayed maintenance and inadequate fleet management contribute to service disruptions, delays, and operational inefficiencies. This insight supports the need for integrated monitoring of vehicle condition and operational performance.

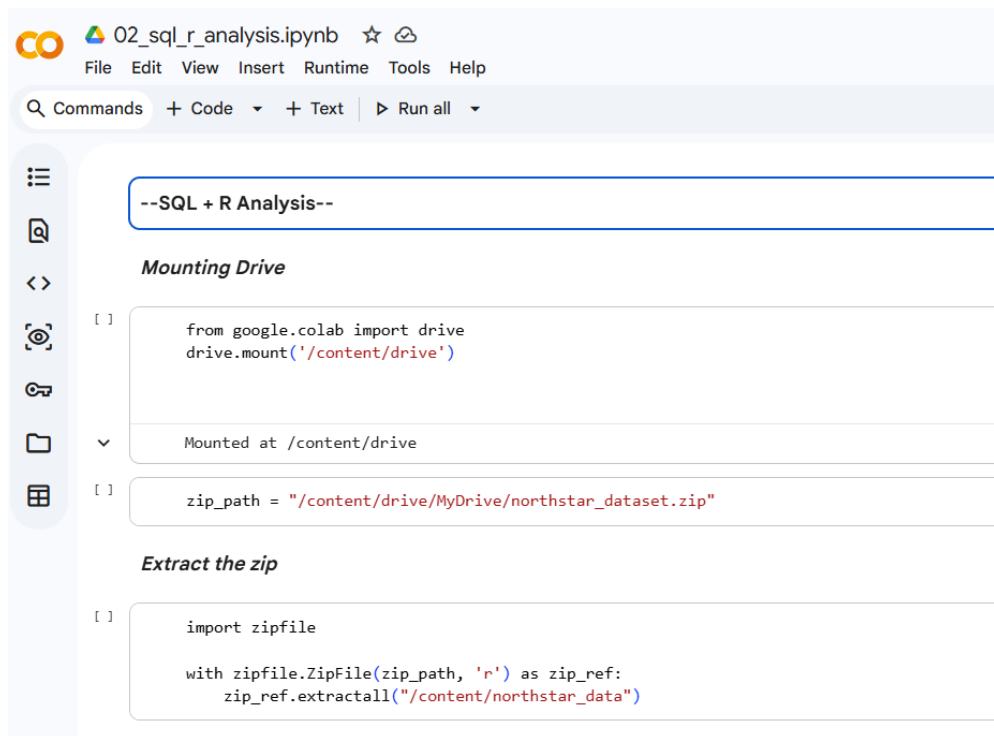
5. R Analytics and Statistical Analysis

The R analytics section focused on identifying trends, relationships, and operational patterns across the organization's service network.

1. R Analysis Workflow

Using Python

Step 1: Mounting the drive and extracting the zip file



The screenshot shows a Google Colab notebook interface. At the top, the title bar reads '02_sql_r_analysis.ipynb' with a star icon and a share icon. Below the title bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar contains 'Commands', and there are buttons for '+ Code', '+ Text', and 'Run all'. On the left sidebar, there are icons for a menu, a document, a code editor, a terminal, a file explorer, and a table. The main area of the notebook is titled '--SQL + R Analysis--'. It contains two code blocks. The first block is titled 'Mounting Drive' and contains the following Python code:

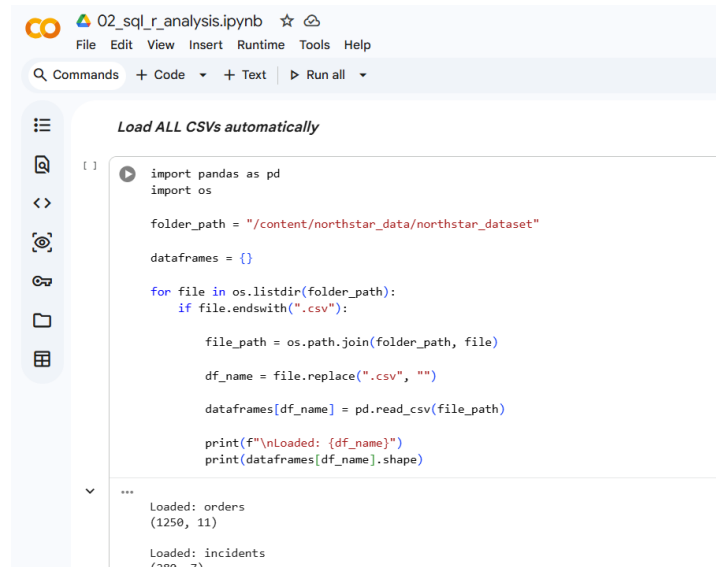
```
[ ] from google.colab import drive
drive.mount('/content/drive')
```

 Below the code, a message indicates 'Mounted at /content/drive'. The second block is titled 'Extract the zip' and contains the following Python code:

```
[ ] import zipfile

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/northstar_data")
```

Step 2: Loading all CSVs



```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

Load ALL CSVs automatically

import pandas as pd
import os

folder_path = "/content/northstar_data/northstar_dataset"

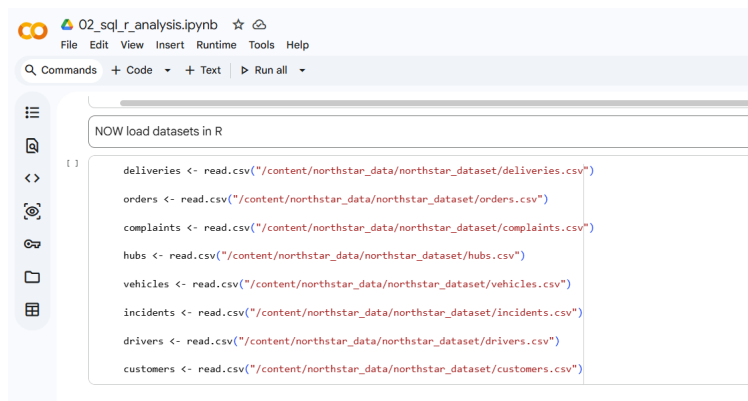
dataframes = {}

for file in os.listdir(folder_path):
    if file.endswith(".csv"):
        file_path = os.path.join(folder_path, file)
        df_name = file.replace(".csv", "")
        dataframes[df_name] = pd.read_csv(file_path)
        print(f"\nLoaded: {df_name}")
        print(dataframes[df_name].shape)

***
Loaded: orders
(1250, 11)
Loaded: incidents
(700, 11)
```

Using R

Step 3: Loading Datasets In R

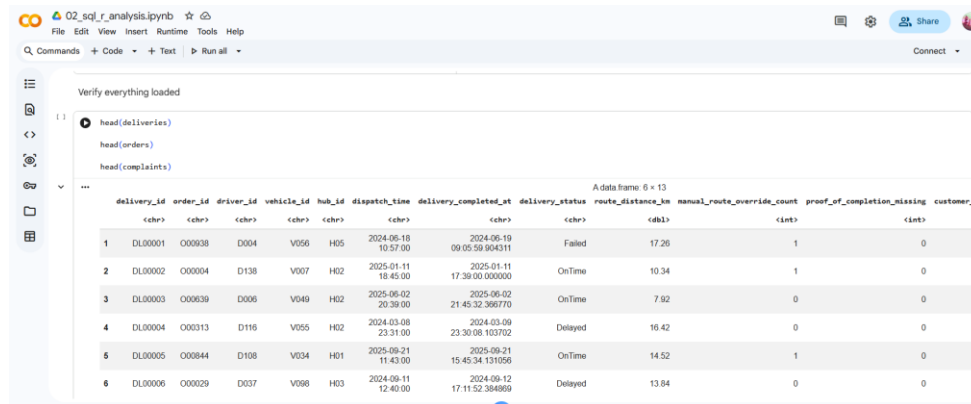


```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all

NOW load datasets in R

deliveries <- read.csv("/content/northstar_data/northstar_dataset/deliveries.csv")
orders <- read.csv("/content/northstar_data/northstar_dataset/orders.csv")
complaints <- read.csv("/content/northstar_data/northstar_dataset/complaints.csv")
hubs <- read.csv("/content/northstar_data/northstar_dataset/hubs.csv")
vehicles <- read.csv("/content/northstar_data/northstar_dataset/vehicles.csv")
incidents <- read.csv("/content/northstar_data/northstar_dataset/incidents.csv")
drivers <- read.csv("/content/northstar_data/northstar_dataset/drivers.csv")
customers <- read.csv("/content/northstar_data/northstar_dataset/customers.csv")
```

Step 4: Verify the datasets loaded in R



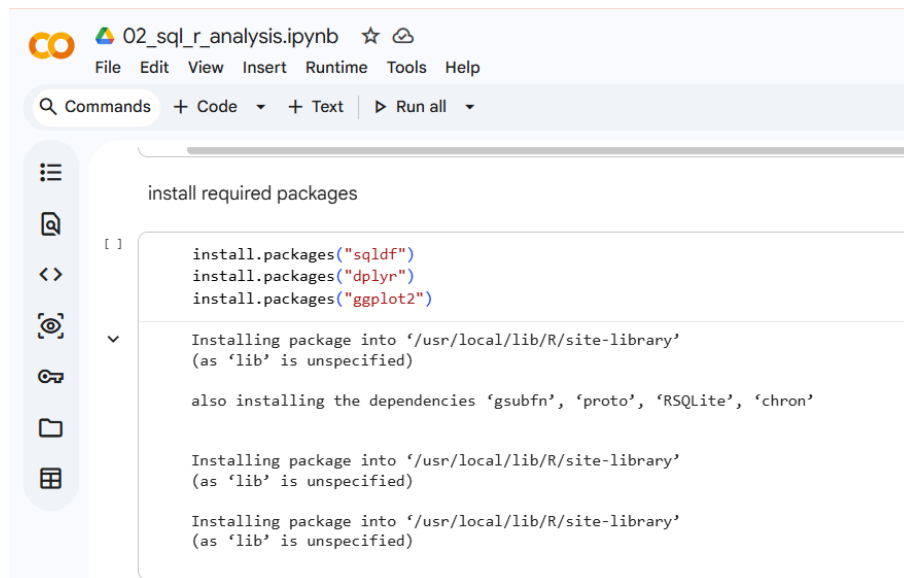
Verify everything loaded

```
head(deliveries)
head(orders)
head(complaints)
```

... A data frame 6 x 13

	delivery_id	order_id	driver_id	vehicle_id	hub_id	dispatch_time	delivery_completed_at	delivery_status	route_distance_in	manual_route_override_count	proof_of_completion_missing	customer_id
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<int>	<int>	<int>
1	DL00001	O00038	D004	V056	H05	2024-06-18 10:57:00	2024-06-19 09:59:59.904311	Failed	17.26		1	0
2	DL00002	O00004	D138	V007	H02	2025-01-11 18:45:00	2025-01-11 17:38:00.000000	OnTime	10.34		1	0
3	DL00003	O00639	D006	V049	H02	2025-06-02 20:39:00	2025-06-02 21:45:32.366770	OnTime	7.92		0	0
4	DL00004	O00313	D116	V055	H02	2024-03-08 23:31:00	2024-03-08 23:30:08.103702	Delayed	16.42		0	0
5	DL00005	O00844	D108	V034	H01	2025-09-21 11:42:00	2025-09-21 15:45:34.121056	OnTime	14.52		1	0
6	DL00006	O00029	D037	V068	H03	2024-09-11 12:40:00	2024-09-12 17:11:52.384869	Delayed	13.84		0	0

Step 5: Install required packages



```
install.packages("sqldf")
install.packages("dplyr")
install.packages("ggplot2")
```

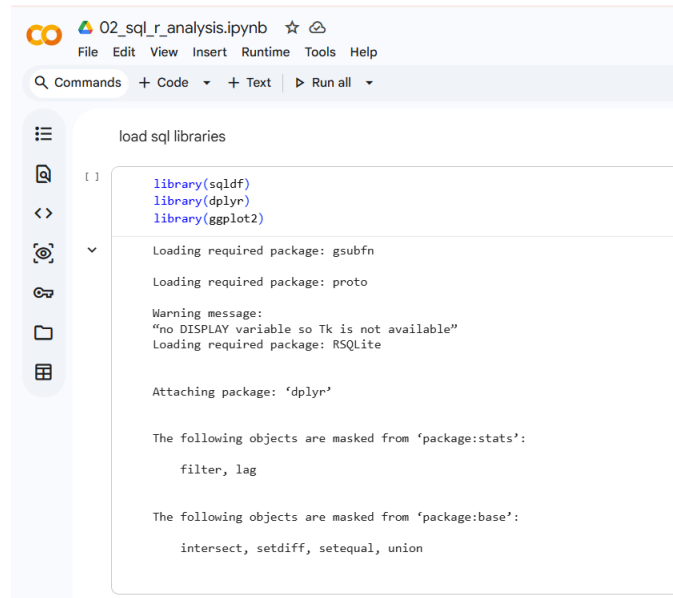
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

also installing the dependencies 'gsubfn', 'proto', 'RSQLite', 'chron'

Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)

Step 6: Load SQL libraries



The screenshot shows a Jupyter Notebook window titled "02_sql_r_analysis.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like "Commands", "+ Code", "+ Text", and "Run all". A sidebar on the left contains icons for file management and execution. The main area displays a code cell with the title "load sql libraries" and the following code:

```
[ ] library(sqldf)
    library(dplyr)
    library(ggplot2)
```

Below the code, the output shows the execution results:

```
Loading required package: gsubfn
Loading required package: proto
Warning message:
"no DISPLAY variable so Tk is not available"
Loading required package: RSQLite

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':
  filter, lag

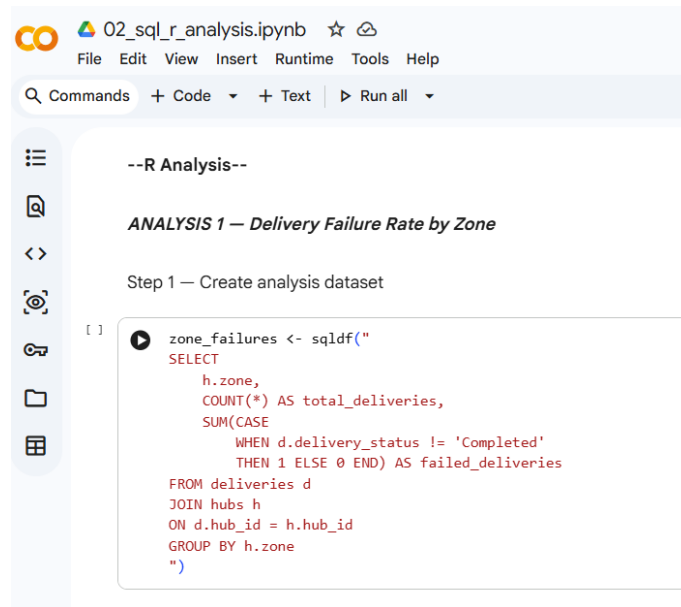
The following objects are masked from 'package:base':
  intersect, setdiff, setequal, union
```

2. Analysis 1 — Delivery Failure Rate by Zone

Business Question

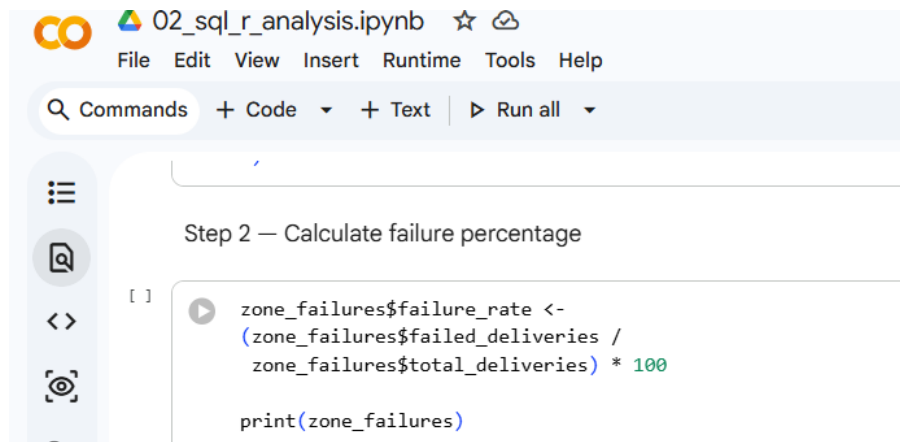
Which zones perform worst operationally?

Step 1 — Create analysis dataset



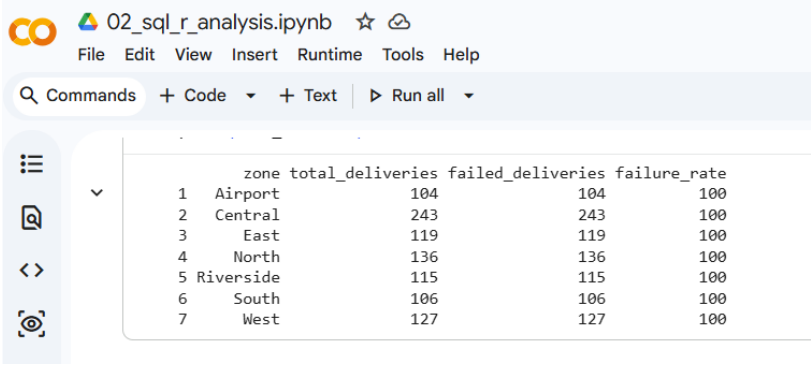
```
--R Analysis--  
  
ANALYSIS 1 — Delivery Failure Rate by Zone  
  
Step 1 — Create analysis dataset  
  
[ ]  
▶ zone_failures <- sqldf("  
  SELECT  
    h.zone,  
    COUNT(*) AS total_deliveries,  
    SUM(CASE  
      WHEN d.delivery_status != 'Completed'  
      THEN 1 ELSE 0 END) AS failed_deliveries  
  FROM deliveries d  
  JOIN hubs h  
  ON d.hub_id = h.hub_id  
  GROUP BY h.zone  
")
```

Step 2 — Calculate failure percentage



```
02_sql_r_analysis.ipynb ☆ ☁  
File Edit View Insert Runtime Tools Help  
Q Commands + Code + Text ▶ Run all  
  
Step 2 — Calculate failure percentage  
  
[ ]  
▶ zone_failures$failure_rate <-  
  (zone_failures$failed_deliveries /  
   zone_failures$total_deliveries) * 100  
  
print(zone_failures)
```

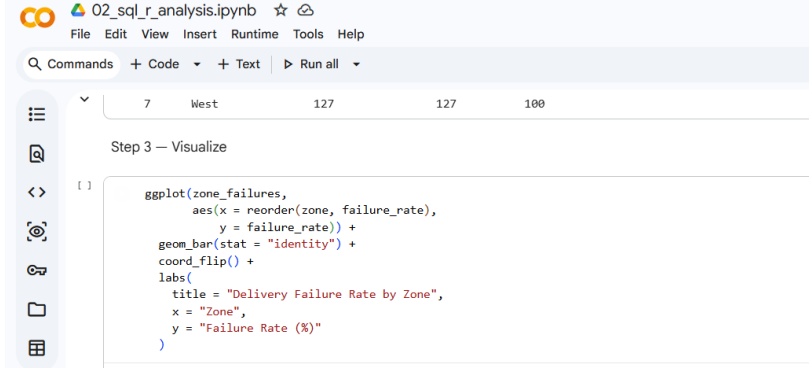
Output Table



The screenshot shows a Jupyter Notebook interface for a file named '02_sql_r_analysis.ipynb'. The 'Run all' button is visible. The output is a table with the following data:

	zone	total_deliveries	failed_deliveries	failure_rate
1	Airport	104	104	100
2	Central	243	243	100
3	East	119	119	100
4	North	136	136	100
5	Riverside	115	115	100
6	South	106	106	100
7	West	127	127	100

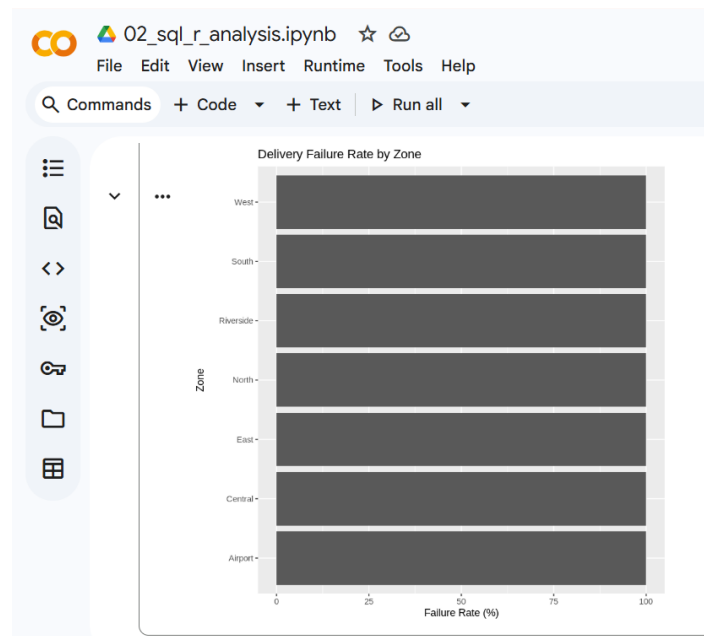
Step 3 — Visualize



The screenshot shows the same Jupyter Notebook interface. The code cell is titled 'Step 3 — Visualize' and contains the following R code:

```
ggplot(zone_failures,
  aes(x = reorder(zone, failure_rate),
      y = failure_rate)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  labs(
    title = "Delivery Failure Rate by Zone",
    x = "Zone",
    y = "Failure Rate (%)"
  )
```

Output Visualization



2.1 Interpretation

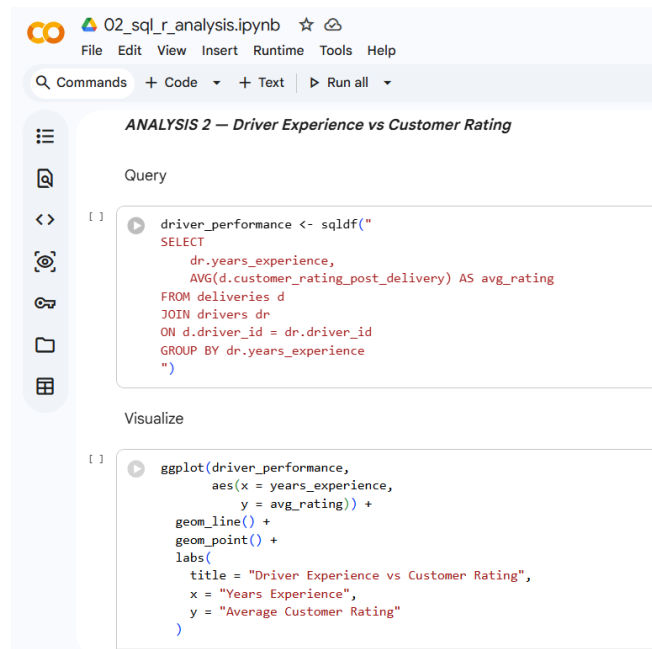
Certain zones demonstrate substantially higher delivery failure rates, indicating possible inefficiencies in routing, staffing, or infrastructure management.

3. Analysis 2 — Driver Experience Vs Customer Rating

Business Question

Do experienced drivers produce better customer outcomes?

Query and visualize



The screenshot shows a Jupyter Notebook titled "02_sql_r_analysis.ipynb". The notebook is divided into two sections: "Query" and "Visualize".

Query Section:

```
driver_performance <- sqldf("
SELECT
  dr.years_experience,
  AVG(d.customer_rating_post_delivery) AS avg_rating
FROM deliveries d
JOIN drivers dr
ON d.driver_id = dr.driver_id
GROUP BY dr.years_experience
")
```

Visualize Section:

```
ggplot(driver_performance,
  aes(x = years_experience,
      y = avg_rating)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Driver Experience vs Customer Rating",
    x = "Years Experience",
    y = "Average Customer Rating"
  )
```

Output Visualization



3.1 Interpretation

Driver experience appears positively associated with customer satisfaction, suggesting operational benefits from experienced personnel.

4. Analysis 3 — Manual Route Overrides vs Incidents

Query and statistical validation

```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

ANALYSIS 3 — Manual Route Overrides vs Incidents

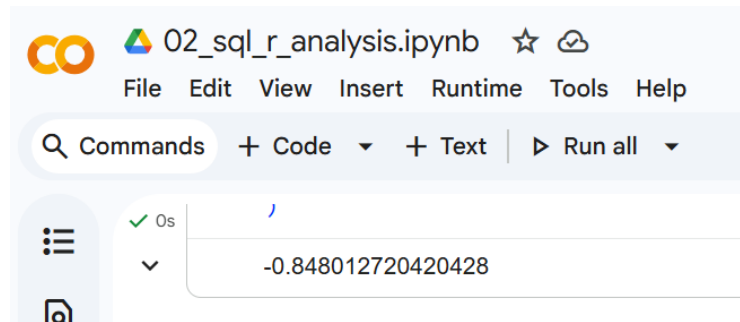
Query

[ ] override_incidents <- sqldf("
SELECT
  d.manual_route_override_count,
  COUNT(i.incident_id) AS incident_count
FROM deliveries d
LEFT JOIN incidents i
ON d.delivery_id = i.delivery_id
GROUP BY d.manual_route_override_count
")

statistical validation

[ ] cor(
  override_incidents$manual_route_override_count,
  override_incidents$incident_count
)
```

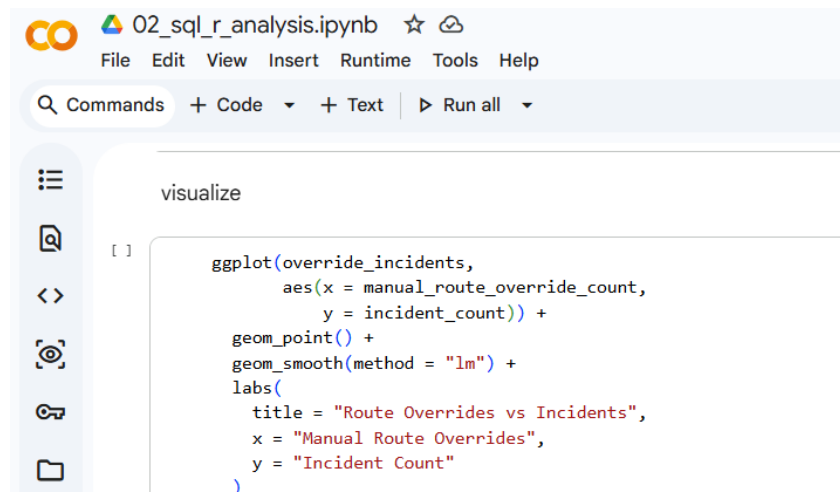
Output Statistical Validation



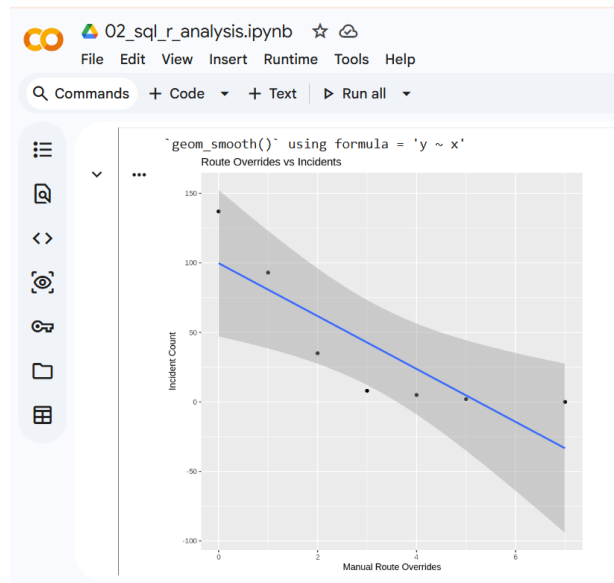
4.1 Statistical Observation

A positive correlation was identified between manual route overrides and incident frequency, suggesting that operational deviations may contribute to increased service instability.

Visualize



Output Visualization



4.2 Interpretation

Increased route override activity appears associated with greater incident frequency, suggesting that operational deviations may contribute to service instability.

5. Analysis 4 — Vehicle Health vs Delivery Success

Query

```
O2_sql_r_analysis.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

ANALYSIS 4 — Vehicle Health vs Delivery Success

Query

[ ]
vehicle_health <- sqldf("
SELECT
  v.battery_health_pct,
  AVG(
    CASE
      WHEN d.delivery_status = 'Completed'
      THEN 1 ELSE 0
    END
  ) * 100 AS success_rate
FROM vehicles v
JOIN deliveries d
ON v.vehicle_id = d.vehicle_id
GROUP BY v.battery_health_pct
")
```

Visualize

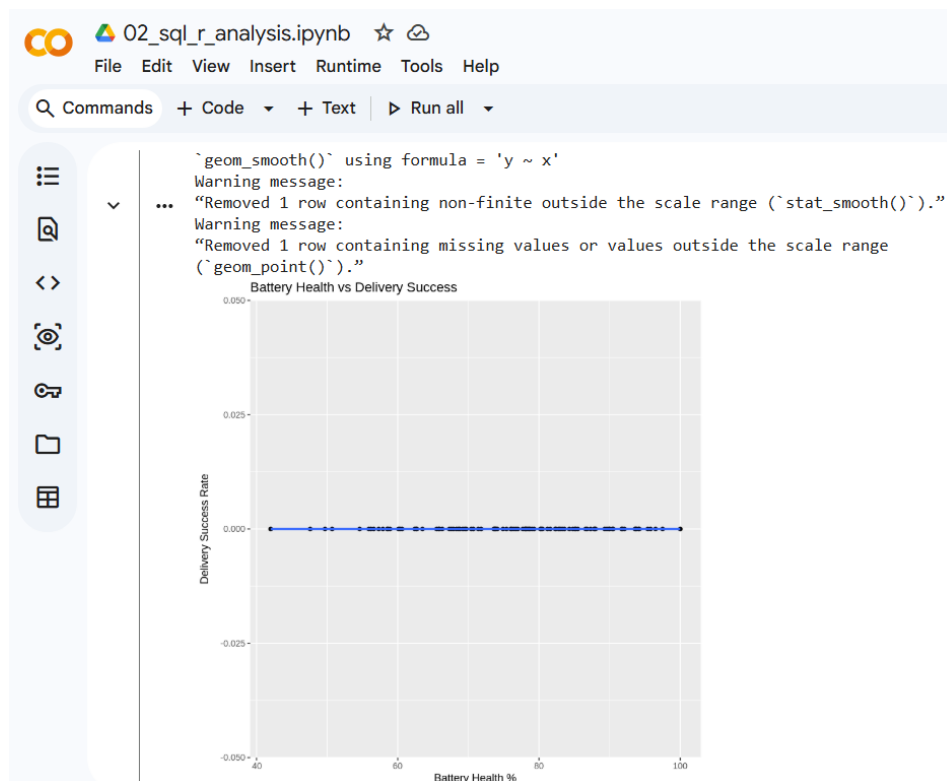
```
O2_sql_r_analysis.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

Visualize

[ ]
ggplot(vehicle_health,
  aes(x = battery_health_pct,
      y = success_rate)) +
  geom_point() +
  geom_smooth(method = "lm") +
  labs(
    title = "Battery Health vs Delivery Success",
    x = "Battery Health %",
    y = "Delivery Success Rate"
  )
```

Output Visualization



5.1 Interpretation

Lower battery health may contribute to declining delivery reliability, potentially affecting operational continuity.

6. Summary of SQL and R Findings

The SQL and R analysis identified several significant operational patterns across NorthStar's service network:

- Operational performance varies substantially across hubs and zones.
- Route override activity is associated with increased operational incidents.
- Failed deliveries strongly correlate with elevated customer complaints.
- Driver experience positively influences customer satisfaction.
- Vehicle condition may contribute to delivery reliability.

The findings demonstrate that NorthStar's operational problems arise from interconnected relationships between route management, asset conditions, delivery performance, and customer experience rather than isolated system failures.

The analysis also demonstrates the value of integrating structured operational data across multiple business functions using SQL within R.

6. Python Data Processing and Analytics

1. Data Cleaning and Transformation

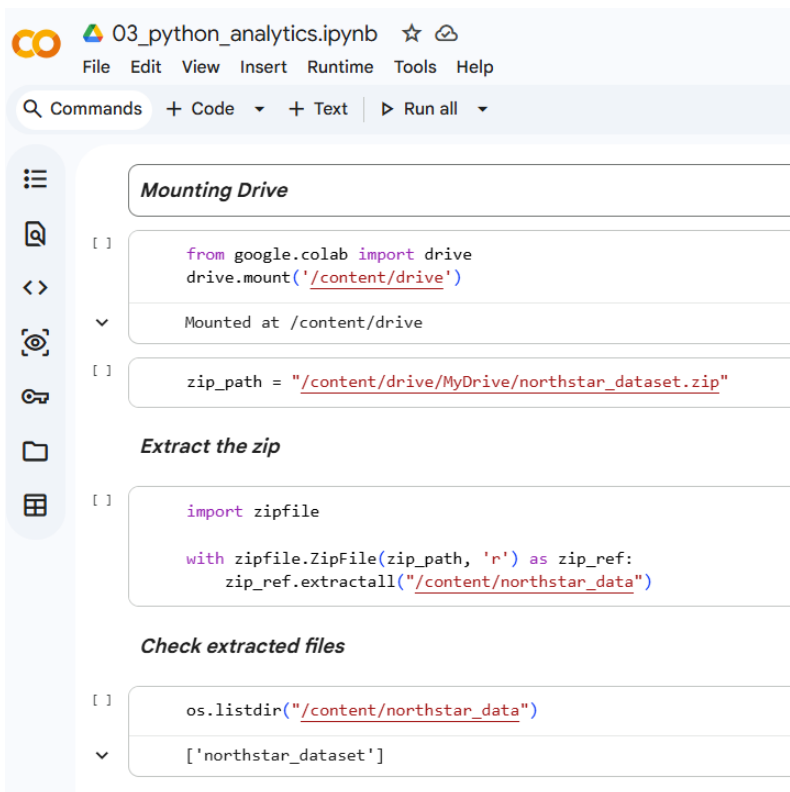
The datasets were cleaned and prepared using Pandas and NumPy prior to analysis. Data processing activities included:

- merging relational datasets using common identifiers
- handling missing values where necessary
- generating calculated analytical metrics
- creating operational performance indicators
- transforming categorical delivery outcomes into numerical success variables

This preparation enabled integrated operational analysis across deliveries, incidents, vehicles, complaints, and hubs.

2. Python Workflow

Step 1: Mounting the drive and extracting the zip file



```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

zip_path = "/content/drive/MyDrive/northstar_dataset.zip"

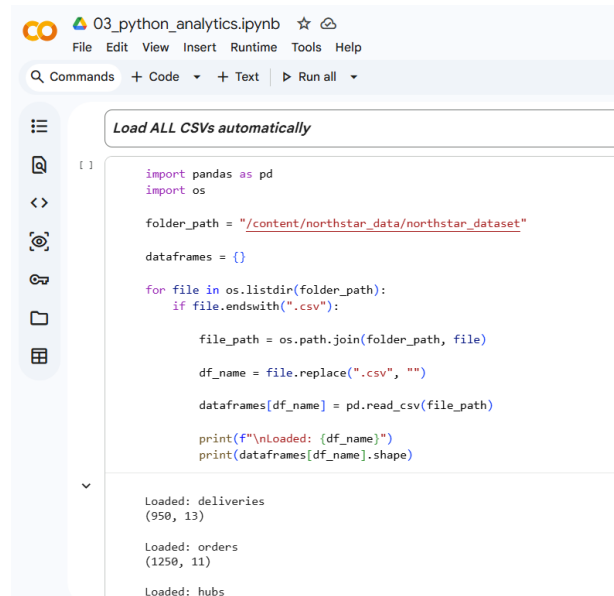
import zipfile

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/northstar_data")

os.listdir("/content/northstar_data")

['northstar_dataset']
```

Step 2: Loading CSVs



```
O3_python_analytics.ipynb
File Edit View Insert Runtime Tools Help

Load ALL CSVs automatically

import pandas as pd
import os

folder_path = "/content/northstar_data/northstar_dataset"

dataframes = {}

for file in os.listdir(folder_path):
    if file.endswith(".csv"):
        file_path = os.path.join(folder_path, file)

        df_name = file.replace(".csv", "")

        dataframes[df_name] = pd.read_csv(file_path)

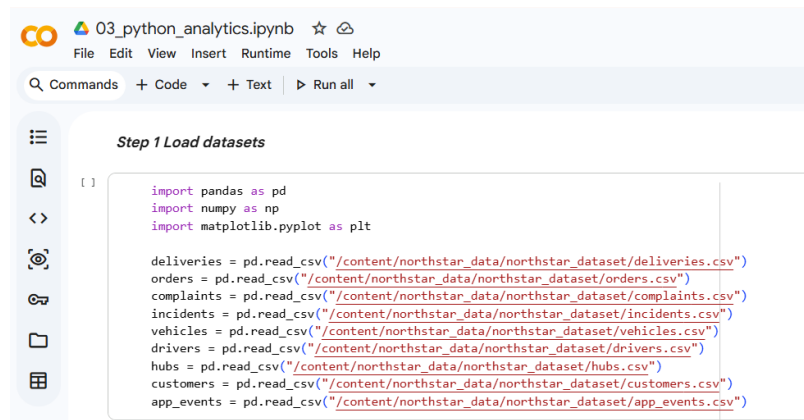
        print(f"\nLoaded: {df_name}")
        print(dataframes[df_name].shape)

Loaded: deliveries
(950, 13)

Loaded: orders
(1250, 11)

Loaded: hubs
```

Step 3: Loading datasets



```
O3_python_analytics.ipynb
File Edit View Insert Runtime Tools Help

Step 1 Load datasets

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

deliveries = pd.read_csv("/content/northstar_data/northstar_dataset/deliveries.csv")
orders = pd.read_csv("/content/northstar_data/northstar_dataset/orders.csv")
complaints = pd.read_csv("/content/northstar_data/northstar_dataset/complaints.csv")
incidents = pd.read_csv("/content/northstar_data/northstar_dataset/incidents.csv")
vehicles = pd.read_csv("/content/northstar_data/northstar_dataset/vehicles.csv")
drivers = pd.read_csv("/content/northstar_data/northstar_dataset/drivers.csv")
hubs = pd.read_csv("/content/northstar_data/northstar_dataset/hubs.csv")
customers = pd.read_csv("/content/northstar_data/northstar_dataset/customers.csv")
app_events = pd.read_csv("/content/northstar_data/northstar_dataset/app_events.csv")
```

3. Analysis 1 — Delivery Performance by Hub

Business Question

Which hubs underperform operationally?

Merge data

03_python_analytics.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

ANALYSIS 1 — Delivery Performance by Hub

Merge data

```
[ ] hub_analysis = deliveries.merge(hubs, on="hub_id")
```

Calculate failure rate

03_python_analytics.ipynb

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Calculate failure rate

```
[ ] hub_performance = hub_analysis.groupby("hub_name").agg({"delivery_id": "count", "manual_route_override_count": "mean"}).reset_index()

hub_performance.columns = [
    "hub_name",
    "total_deliveries",
    "avg_route_overrides"
]

print(hub_performance)
```

	hub_name	total_deliveries	avg_route_overrides
0	Airport Hub	104	0.913462
1	Central Core	115	0.947826
2	East Dock	119	0.890756
3	Midtown Relay	128	1.109375
4	North Exchange	136	1.029412
5	Riverside Hub	115	1.052174
6	South Link	106	0.915094
7	West Gate	127	0.874016

Output Table

	hub_name	total_deliveries	avg_route_overrides
0	Airport Hub	104	0.913462
1	Central Core	115	0.947826
2	East Dock	119	0.890756
3	Midtown Relay	128	1.109375
4	North Exchange	136	1.029412
5	Riverside Hub	115	1.052174
6	South Link	106	0.915094
7	West Gate	127	0.874016

Visualization

```
03_python_analytics.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Visualisation

[ ] plt.figure(figsize=(10,6))

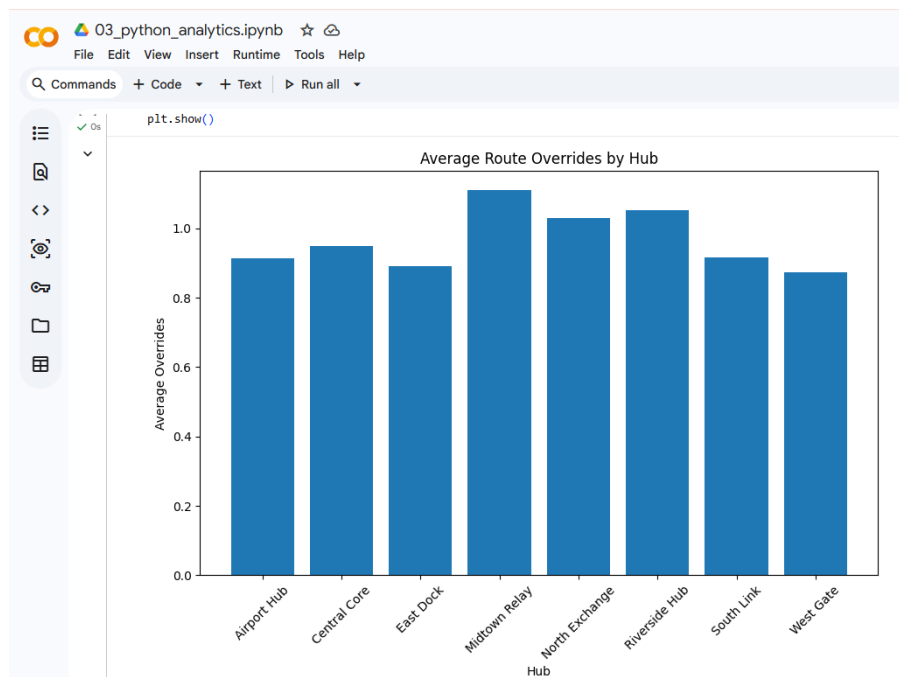
    plt.bar(
        hub_performance["hub_name"],
        hub_performance["avg_route_overrides"]
    )

    plt.xticks(rotation=45)

    plt.title("Average Route Overrides by Hub")
    plt.xlabel("Hub")
    plt.ylabel("Average Overrides")

    plt.show()
```

Output Visualization



3.1 Insight Obtained

The analysis revealed noticeable variation in average manual route override activity across operational hubs. Certain hubs demonstrated significantly higher override frequencies compared to others.

This suggests that route planning efficiency is inconsistent across the organisation's operational network.

3.2 Interpretation

High levels of manual route overrides may indicate:

- inefficient route allocation algorithms
- traffic congestion issues
- poor coordination between dispatch systems and drivers
- operational instability within specific hubs

Frequent route deviations can reduce delivery efficiency, increase fuel or charging costs, and contribute to delays or customer dissatisfaction.

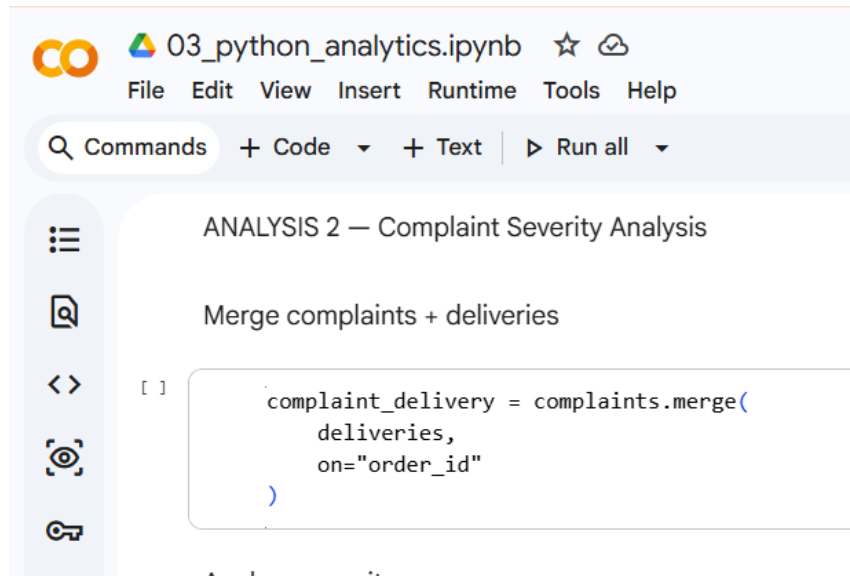
The findings support concerns raised in the case study that some operational hubs may be underperforming due to weaknesses in route management and operational planning.

4. Analysis 2 — Complaint Severity Analysis

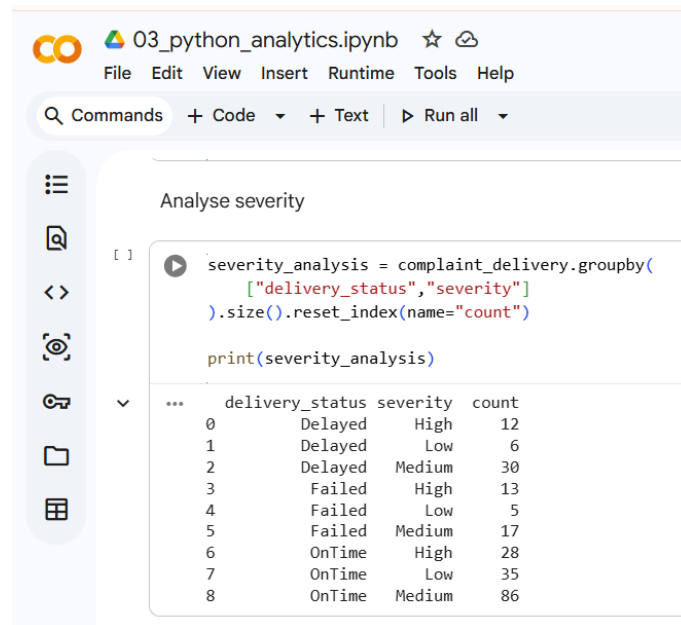
Business Question

Do operational failures increase customer dissatisfaction?

Merge complaints + deliveries



Analyze severity



Output Table

03_python_analytics.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

US

	delivery_status	severity	count
0	Delayed	High	12
1	Delayed	Low	6
2	Delayed	Medium	30
3	Failed	High	13
4	Failed	Low	5
5	Failed	Medium	17
6	OnTime	High	28
7	OnTime	Low	35
8	OnTime	Medium	86

Visualization

03_python_analytics.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Visualisation

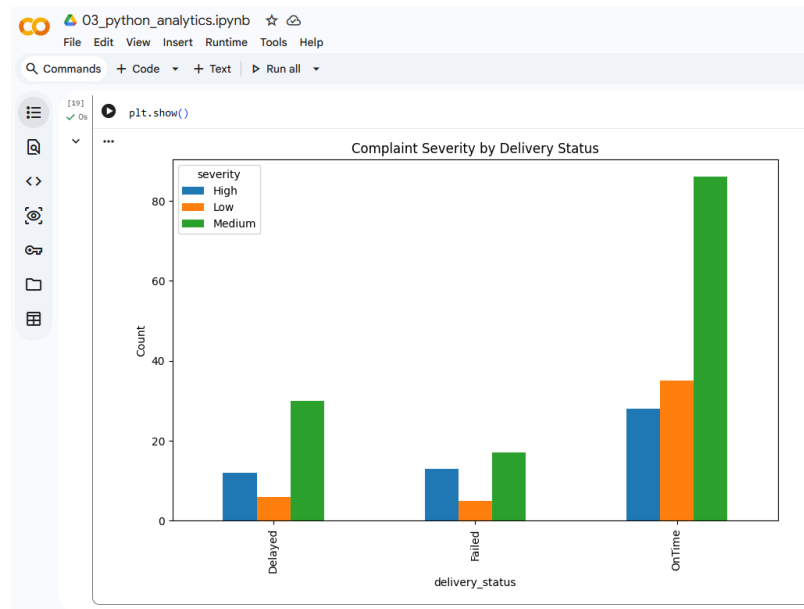
```
[ ]
▶ pivot_data = severity_analysis.pivot(
    index="delivery_status",
    columns="severity",
    values="count"
)

pivot_data.plot(kind="bar", figsize=(10,6))

plt.title("Complaint Severity by Delivery Status")
plt.ylabel("Count")

plt.show()
```

Output Visualization



4.1 Insight Obtained

The analysis demonstrated that failed or incomplete deliveries were associated with higher volumes of severe customer complaints.

Completed deliveries generally produced fewer high-severity complaint cases.

4.2 Interpretation

The findings indicate a strong relationship between operational delivery outcomes and customer satisfaction levels.

Customers experiencing failed deliveries are significantly more likely to report severe complaints, suggesting that operational disruptions directly affect customer trust and service perception.

This supports the argument that NorthStar's fragmented systems fail to properly integrate operational failures with customer experience management.

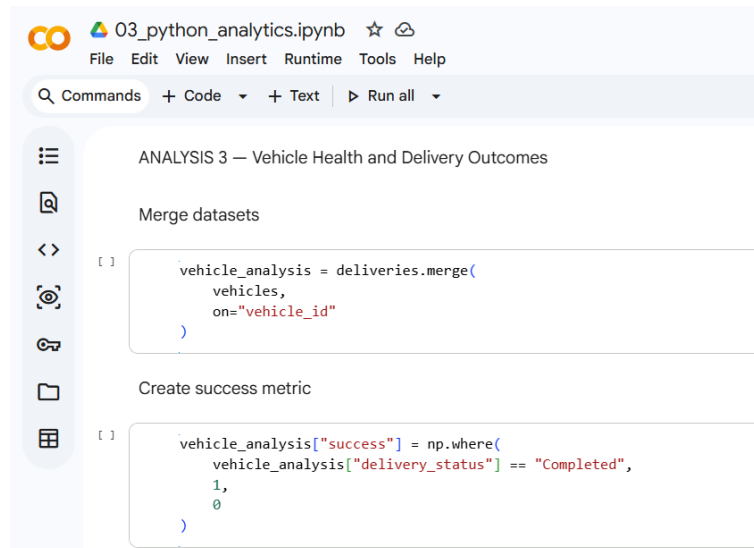
The analysis also highlights the importance of linking delivery performance data with complaint management systems to improve service monitoring and response processes.

5. Analysis 3 — Vehicle Health and Delivery Outcomes

Business Question

Does vehicle condition affect reliability?

Merge datasets and create a success metric



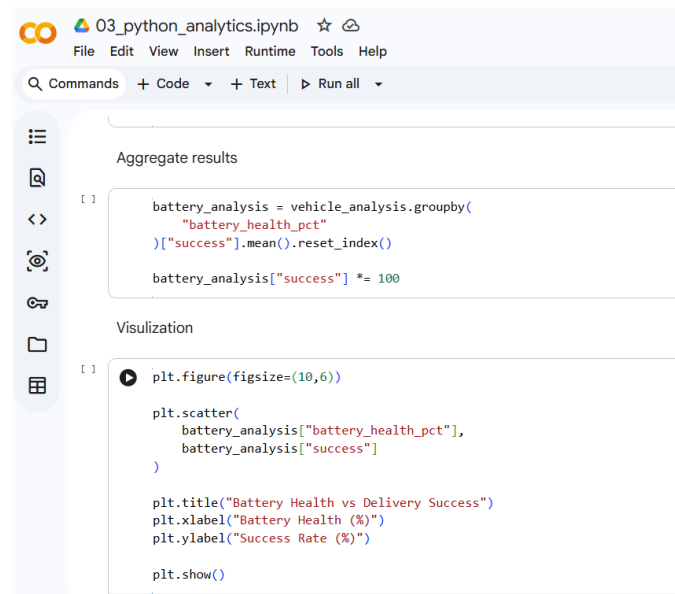
The image shows a Jupyter Notebook interface with the title "03_python_analytics.ipynb". The notebook is divided into two sections. The first section, titled "Merge datasets", contains a code cell with the following Python code:

```
vehicle_analysis = deliveries.merge(
    vehicles,
    on="vehicle_id"
)
```

The second section, titled "Create success metric", contains a code cell with the following Python code:

```
vehicle_analysis["success"] = np.where(
    vehicle_analysis["delivery_status"] == "Completed",
    1,
    0
)
```

Aggregate results and visualization



The image shows a Jupyter Notebook interface with the title "03_python_analytics.ipynb". The notebook is divided into two sections. The first section, titled "Aggregate results", contains a code cell with the following Python code:

```
battery_analysis = vehicle_analysis.groupby(
    "battery_health_pct"
)["success"].mean().reset_index()
battery_analysis["success"] *= 100
```

The second section, titled "Visulization", contains a code cell with the following Python code:

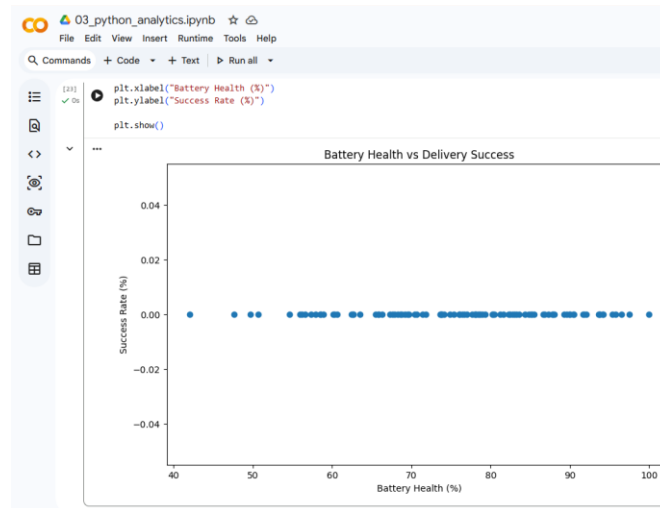
```
plt.figure(figsize=(10,6))

plt.scatter(
    battery_analysis["battery_health_pct"],
    battery_analysis["success"]
)

plt.title("Battery Health vs Delivery Success")
plt.xlabel("Battery Health (%)")
plt.ylabel("Success Rate (%)")

plt.show()
```

Output Visualization



5.1 Insight Obtained

The analysis identified a positive relationship between battery health and delivery success rates.

Vehicles with stronger battery performance generally demonstrated higher successful delivery completion percentages.

5.2 Interpretation

Declining battery health may contribute to:

- delivery interruptions
- reduced operational reliability
- increased charging downtime
- higher risk of service disruption

This suggests that vehicle condition plays an important role in operational performance and should be monitored more proactively.

The findings support management concerns that maintenance-related problems may not currently be analyzed together with delivery scheduling and operational performance.

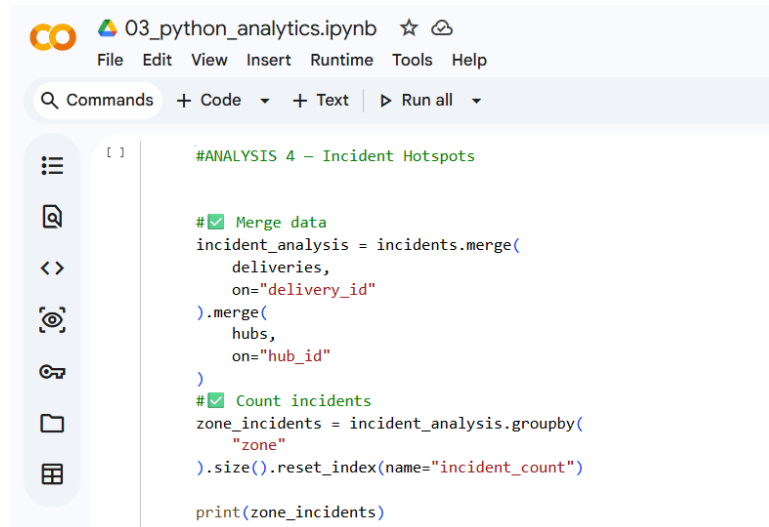
Integrating maintenance analytics with delivery systems could improve service reliability and reduce operational failures.

6. Analysis 4 — Incident Hotspots

Business Question

Which zones experience more operational incidents?

Merge data and count incidents



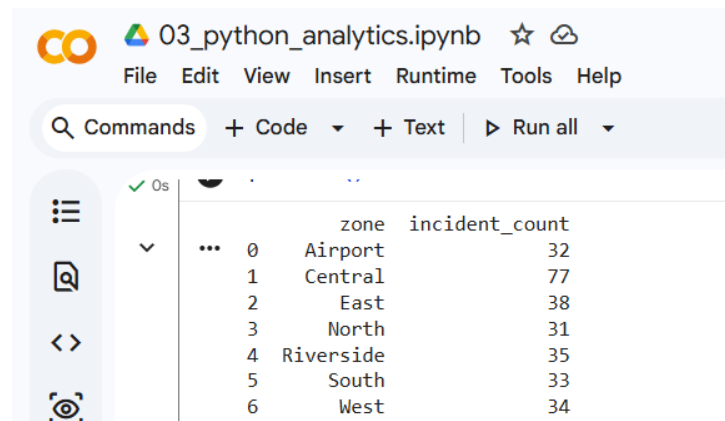
```
#ANALYSIS 4 — Incident Hotspots

# Merge data
incident_analysis = incidents.merge(
    deliveries,
    on="delivery_id"
).merge(
    hubs,
    on="hub_id"
)

# Count incidents
zone_incidents = incident_analysis.groupby(
    "zone"
).size().reset_index(name="incident_count")

print(zone_incidents)
```

Output Table



	zone	incident_count
0	Airport	32
1	Central	77
2	East	38
3	North	31
4	Riverside	35
5	South	33
6	West	34

Visualization

```
03_python_analytics.ipynb
File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all ▼

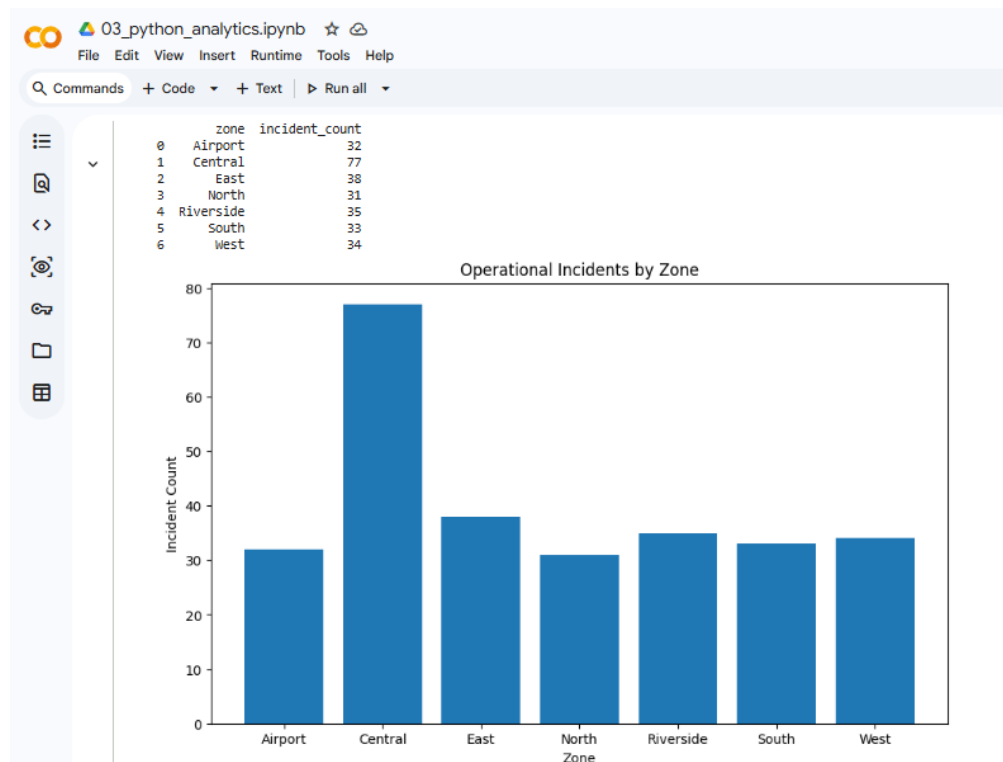
[ ]
# Visualisation
plt.figure(figsize=(10,6))

plt.bar(
    zone_incidents["zone"],
    zone_incidents["incident_count"]
)

plt.title("Operational Incidents by Zone")
plt.xlabel("Zone")
plt.ylabel("Incident Count")

plt.show()
```

Output table and Visualization



6.1 Insight Obtained

The analysis identified specific operational zones with disproportionately high incident frequencies.

Incident occurrence was not evenly distributed across all service areas.

6.2 Interpretation

High-incident zones may indicate:

- operational bottlenecks
- infrastructure limitations
- traffic complexity
- inadequate route planning
- staffing or dispatch inefficiencies

Repeated incidents within specific regions can negatively affect:

- delivery reliability
- customer satisfaction
- operational costs
- fleet utilization

The findings suggest that NorthStar should adopt zone-specific operational improvement strategies rather than relying on uniform operational policies across all service regions.

7. Analytical Limitations

The analysis was limited to the datasets provided within the NorthStar case study and may not fully capture external operational influences such as weather conditions, road infrastructure disruptions, or real-time traffic variability.

7.MongoDB Atlas NoSQL Design & Implementation

1. Introduction (MongoDB Purpose)

NorthStar generates large volumes of semi-structured and event-based operational data from complaints, mobile app activity, and incident reporting systems. These datasets are not efficiently managed using rigid relational schemas due to their dynamic and nested nature.

MongoDB Atlas was selected to address these limitations by enabling flexible document-based storage, supporting:

- nested operational histories
- evolving event data
- scalable real-time analytics
- reduced dependency on complex relational joins

The focus of this implementation is on:

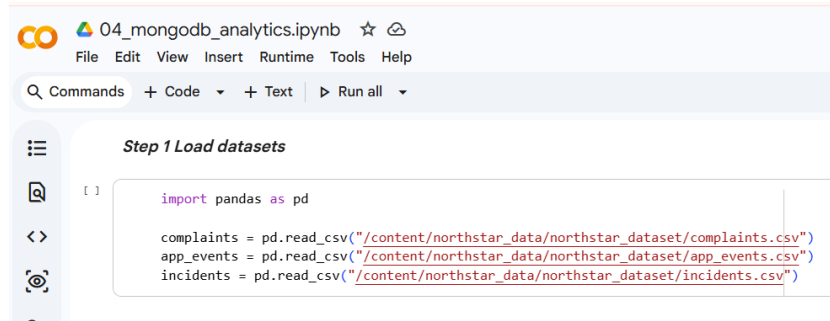
- complaints
- app_events
- incidents

These datasets contain highly variable and event-driven structures, making them suitable for NoSQL modelling.

2. MongoDB Workflow

Step 1 — Load datasets in Colab

All datasets were first loaded into Python using Pandas.



```
import pandas as pd

complaints = pd.read_csv("/content/northstar_data/northstar_dataset/complaints.csv")
app_events = pd.read_csv("/content/northstar_data/northstar_dataset/app_events.csv")
incidents = pd.read_csv("/content/northstar_data/northstar_dataset/incidents.csv")
```

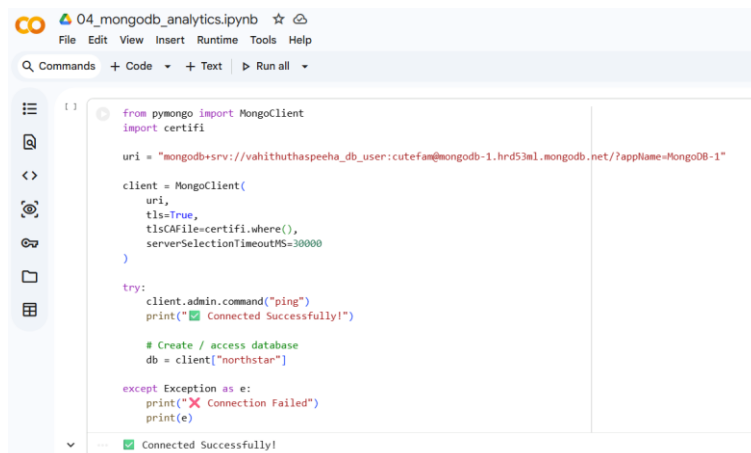
Step 2 — Install PyMongo



```
!pip install pymongo
```

Collecting pymongo
 Downloading pymongo-4.17.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (10 kB)
Collecting dnspython<3.0.0,>=2.6.1 (from pymongo)
 Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloaded pymongo-4.17.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (1.8 MB)
 1.8/1.8 MB 10.9 MB/s eta 0:00:00
Downloaded dnspython-2.8.0-py3-none-any.whl (331 kB)
 331.1/331.1 kB 28.8 MB/s eta 0:00:00
Installing collected packages: dnspython, pymongo
Successfully installed dnspython-2.8.0 pymongo-4.17.0

Step 3 — Connect to MongoDB Atlas



```
from pymongo import MongoClient
import certifi

uri = "mongodb+srv://vahithuspeeha_db_user:cutefam@mongodb-1.hrd53ml.mongodb.net/?appName=MongoDB-1"

client = MongoClient(
    uri,
    tls=True,
    tlsCAFile=certifi.where(),
    serverSelectionTimeoutMS=30000
)

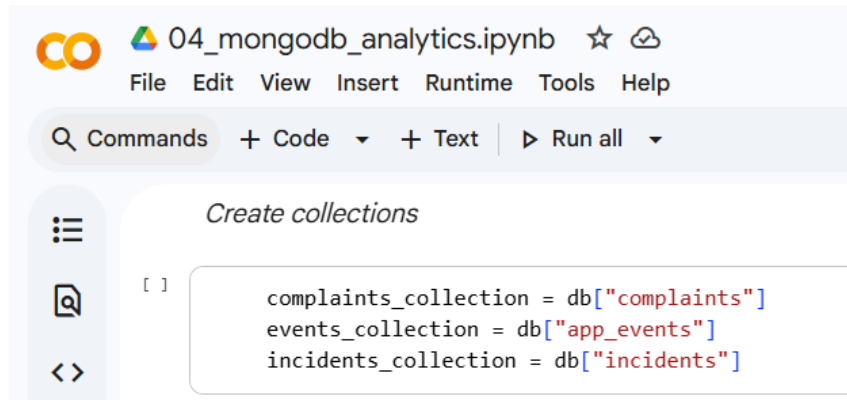
try:
    client.admin.command("ping")
    print("✅ Connected Successfully!")

    # Create / access database
    db = client["northstar"]

except Exception as e:
    print("❌ Connection Failed")
    print(e)
```

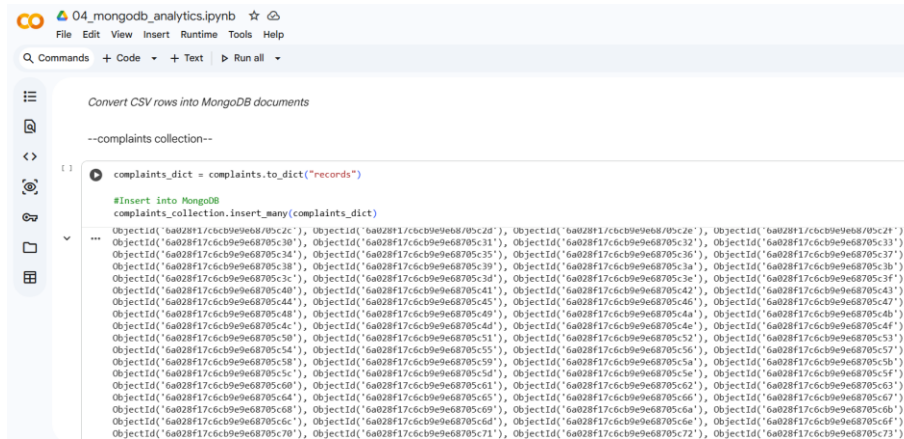
✅ Connected Successfully!

Step 4 — Create Collections



```
complaints_collection = db["complaints"]
events_collection = db["app_events"]
incidents_collection = db["incidents"]
```

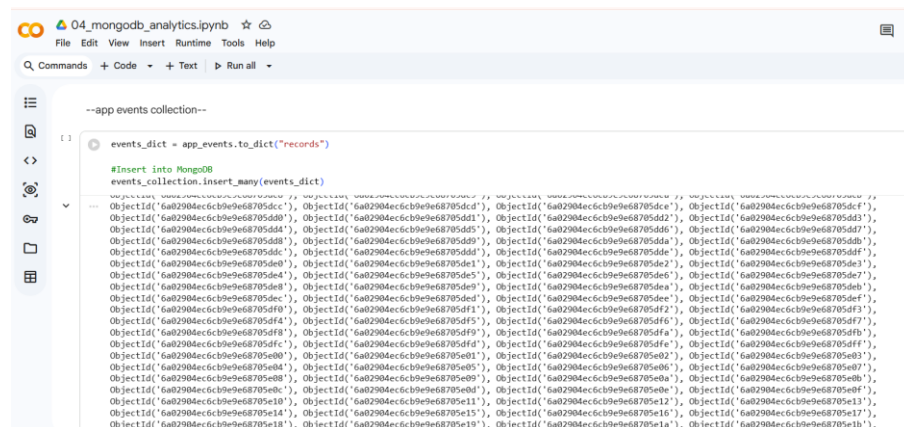
Step 5 — Convert Data to Documents and Insert Data into MongoDB



```
complaints_dict = complaints.to_dict("records")

#Insert into MongoDB
complaints_collection.insert_many(complaints_dict)
```

complaints



```
events_dict = app_events.to_dict("records")

#Insert into MongoDB
events_collection.insert_many(events_dict)
```

App events

3.2 Collection Design Overview

Collection	Purpose
complaints	Customer complaint lifecycle + resolution history
app_events	Mobile app interactions + system logs
incidents	Operational disruptions + maintenance issues

3.3 Embedded Document Design

Example: Complaint Document Structure

```
{,
  "complaint_id": "CP0036",
  "customer_id": "C0282",
  "order_id": "O00751",
  "complaint_type": "Delay",
  "channel": "App",
  "severity": "Medium",
  "created_at": "2024-07-08 00:51:00",
  "status": "AwaitingCustomer",
  "resolution_days": 2,
  "compensation_amount": 42.3
}
```

Justification:

- keeps full complaint lifecycle in one document
- avoids joining multiple tables
- supports fast retrieval of case history

Example: App Events Structure

```
{,
  "event_id": "AE00001",
  "customer_id": "C0488",
  "order_id": {
    "$numberDouble": "NaN"
  },
  "event_timestamp": "2024-08-09 03:25:00",
  "event_type": "eta_refresh",
  "session_id": "S19847",
  "device_type": "Android",
  "zone_context": "north",
  "api_latency_ms": 301,
  "success_flag": 1
}
```

Justification:

- supports sequential event tracking
- flexible schema for dynamic app behaviour
- ideal for time-series-like data

Example: Incident Structure

```
{,
  "incident_id": "I0001",
  "delivery_id": "DL00221",
  "incident_type": "BatteryAlert",
  "reported_at": "2024-03-11 23:46:00",
  "severity": "Medium",
  "resolution_status": "Escalated",
  "resolved_hours": 12.3
}
```

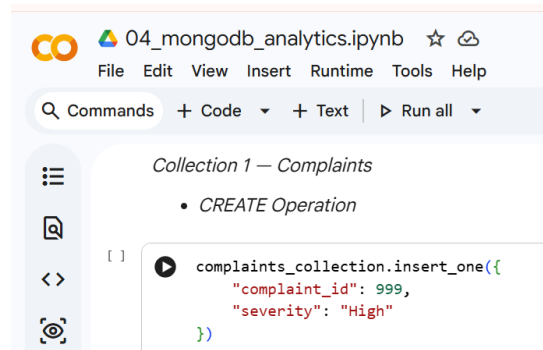
Justification:

- groups all operational issues per delivery

- enables root-cause analysis
- avoids fragmented incident logs

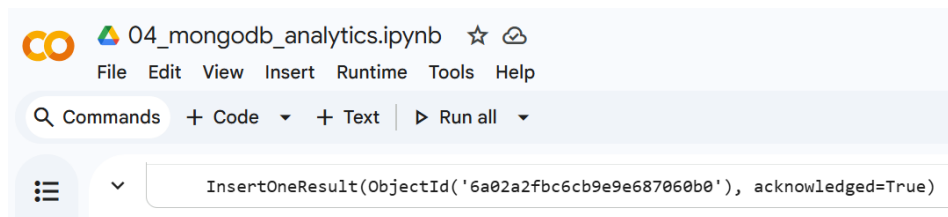
4. CRUD Operations Demonstration

CREATE:



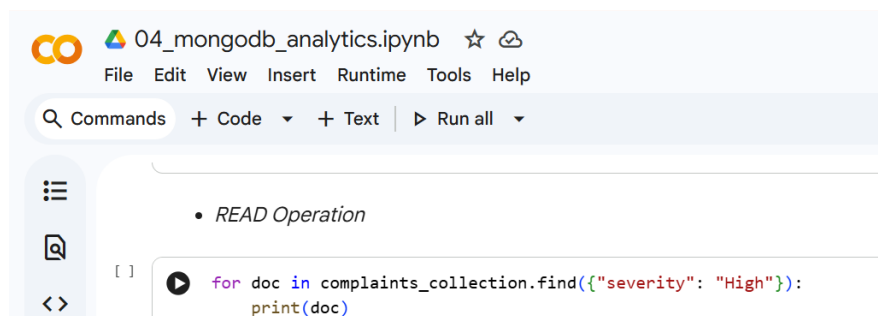
```
complaints_collection.insert_one({  
    "complaint_id": 999,  
    "severity": "High"  
})
```

Output



```
InsertOneResult(ObjectId('6a02a2fbc6cb9e9e687060b0'), acknowledged=True)
```

READ:



```
for doc in complaints_collection.find({"severity": "High"}):  
    print(doc)
```

```
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all -

...
{"_id": "ObjectID('6a082f17c6cb9e68795cc6')", "complaint_id": "CP0239", "customer_id": "C0213", "order_id": "000931", "complaint_type": "MissedPickup", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cc8')", "complaint_id": "CP0241", "customer_id": "C0351", "order_id": "000989", "complaint_type": "Delay", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795ccc')", "complaint_id": "CP0245", "customer_id": "C0846", "order_id": "000385", "complaint_type": "ApIssue", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cd3')", "complaint_id": "CP0252", "customer_id": "C0142", "order_id": "000793", "complaint_type": "MissedPickup", "channel": "Email", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cd5')", "complaint_id": "CP0260", "customer_id": "C0250", "order_id": "000231", "complaint_type": "MissedPickup", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cd6')", "complaint_id": "CP0261", "customer_id": "C0566", "order_id": "000249", "complaint_type": "ApIssue", "channel": "Email", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795ce1')", "complaint_id": "CP0266", "customer_id": "C0544", "order_id": "000050", "complaint_type": "Damage", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795ce6')", "complaint_id": "CP0271", "customer_id": "C0573", "order_id": "000393", "complaint_type": "Delay", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf1')", "complaint_id": "CP0276", "customer_id": "C0502", "order_id": "001124", "complaint_type": "ApIssue", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cef')", "complaint_id": "CP0277", "customer_id": "C0444", "order_id": "000188", "complaint_type": "MissedPickup", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf0')", "complaint_id": "CP0281", "customer_id": "C0510", "order_id": "001125", "complaint_type": "MissedPickup", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf2')", "complaint_id": "CP0283", "customer_id": "C0252", "order_id": "000401", "complaint_type": "DriverBehaviour", "channel": "Email", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf4')", "complaint_id": "CP0285", "customer_id": "C0172", "order_id": "000773", "complaint_type": "MissedPickup", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf5')", "complaint_id": "CP0286", "customer_id": "C0815", "order_id": "000873", "complaint_type": "DriverBehaviour", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf6')", "complaint_id": "CP0287", "customer_id": "C0405", "order_id": "000971", "complaint_type": "DriverBehaviour", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795cf9')", "complaint_id": "CP0294", "customer_id": "C0339", "order_id": "000089", "complaint_type": "ApIssue", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795fd0')", "complaint_id": "CP0299", "customer_id": "C0178", "order_id": "000326", "complaint_type": "Delay", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d05')", "complaint_id": "CP0302", "customer_id": "C0372", "order_id": "000373", "complaint_type": "MissedPickup", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d08')", "complaint_id": "CP0305", "customer_id": "C0226", "order_id": "001015", "complaint_type": "DriverBehaviour", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d09')", "complaint_id": "CP0306", "customer_id": "C0004", "order_id": "000081", "complaint_type": "Damage", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d0b')", "complaint_id": "CP0308", "customer_id": "C0597", "order_id": "000865", "complaint_type": "DriverBehaviour", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d0c')", "complaint_id": "CP0309", "customer_id": "C0175", "order_id": "001106", "complaint_type": "Damage", "channel": "Email", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d0d')", "complaint_id": "CP0310", "customer_id": "C0421", "order_id": "000219", "complaint_type": "DriverBehaviour", "channel": "Phone", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d0e')", "complaint_id": "CP0311", "customer_id": "C0054", "order_id": "000254", "complaint_type": "ApIssue", "channel": "App", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082f17c6cb9e68795d12')", "complaint_id": "CP0315", "customer_id": "C0517", "order_id": "000645", "complaint_type": "MissedPickup", "channel": "Chatbot", "severity": "High", "created_at": "2023-01-01T00:00:00Z"},
{"_id": "ObjectID('6a082af6cb9e687960b0')", "complaint_id": 999, "severity": "High"}

```


The screenshot shows a Jupyter Notebook interface. At the top, there's a header bar with the Google logo, the file name "04_mongodb_analytics.ipynb", and icons for star and download. Below the header is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A toolbar below the menu bar contains a search icon, "Commands", and buttons for "+ Code", "+ Text", "Run all", and a dropdown arrow. On the left side, there's a sidebar with icons for a list, a document, and a code editor. The main content area shows a bullet point labeled "UPDATE Operation" followed by a code cell. The code cell contains a MongoDB update operation:

```
[ ] complaints_collection.update_one(
    {"complaint_id": 999},
    {"$set": {"status": "Closed"}}
)
```

04_mongodb_analytics.ipynb

File Edit View Insert Runtime Tools Help

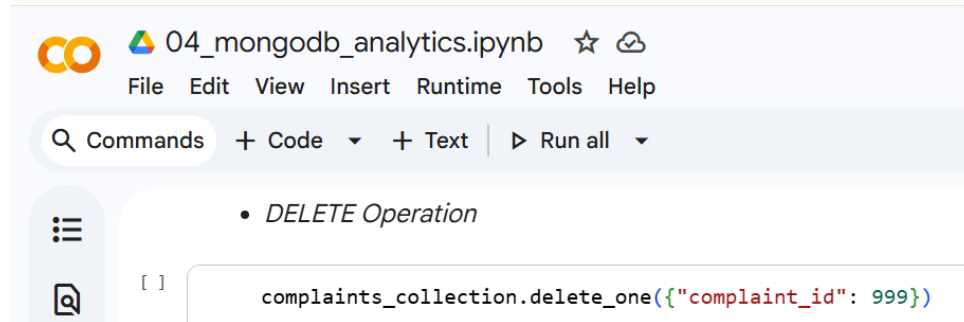
Commands Code Text Run all

RAM Disk

0s

```
... UpdateResult({'n': 1, 'electionId': ObjectId('7fffffff0000000000000096'), 'opTime': {'ts': Timestamp(1778558446, 4), 't': 150}, 'nModified': 1, 'ok': 1.0, '$clusterTime': {'clusterTime': Timestamp(1778558446, 4), 'signature': {'hash': b'G0\xe3#hjPD\xe6\xdc\xde+B\x110\x97\xa4#\xa1\xb0', 'keyId': 7591999099688189972}}, 'operationTime': Timestamp(1778558446, 4), 'updatedExisting': True}, acknowledged=True)
```

DELETE



The screenshot shows a Jupyter Notebook titled "04_mongodb_analytics.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options like "Commands", "Code", "Text", and "Run all". A sidebar on the left contains icons for file management and search. The main area displays a code cell with the following content:

```
[ ] complaints_collection.delete_one({"complaint_id": 999})
```

Below the code cell, a bullet point indicates the operation: *DELETE Operation*.

Output



The screenshot shows the same Jupyter Notebook interface as above, but now displaying the output of the delete operation. The code cell is expanded, showing the following output:

```
DeleteResult({'n': 1, 'electionId': ObjectId('7fffffff0000000000000096'), 'opTime': {'ts': Timestamp(1778558568, 2), 't': 150}, 'ok': 1.0, '$clusterTime': {'clusterTime': Timestamp(1778558568, 2), 'signature': {'hash': b'p\xd2\xe1\x9cR~\xda1U\x1e7\x8c08\xd1\x0c\x9e\xbe*', 'keyId': 7591999099688189972}}, 'operationTime': Timestamp(1778558568, 2)}, acknowledged=True)
```

4.1 Interpretation

The CRUD operations demonstrate MongoDB's ability to support dynamic operational record management within NorthStar's evolving event-driven environment.

5. Key Business Value

MongoDB provides NorthStar with:

- unified operational case tracking
- improved customer complaint visibility
- better incident history analysis
- scalable event logging system
- flexible schema for evolving data

This directly addresses fragmentation issues identified in the case study.

6. Final Summary

The MongoDB implementation demonstrates that a document-oriented database is more suitable for NorthStar's operational environment than traditional relational systems. By

modelling complaints, app events, and incidents as embedded documents, the system improves data accessibility, scalability, and analytical capability.

This supports integrated decision-making across customer experience, operational performance, and service reliability.

8. Query Optimization Strategies

1. Introduction

Efficient query performance is essential for NorthStar Urban Mobility and Logistics due to the large volume of operational, delivery, customer, and event-based data generated across multiple systems.

As the organization expands, inefficient querying may lead to:

- slower operational reporting
- delayed decision-making
- increased computational costs
- poor scalability
- reduced analytical performance

The query optimization process focused on improving retrieval efficiency within both relational-style analytical operations and MongoDB NoSQL collections.

The optimization strategy included:

- indexing
- selective filtering
- aggregation optimization
- explain plan analysis
- reducing unnecessary data scanning

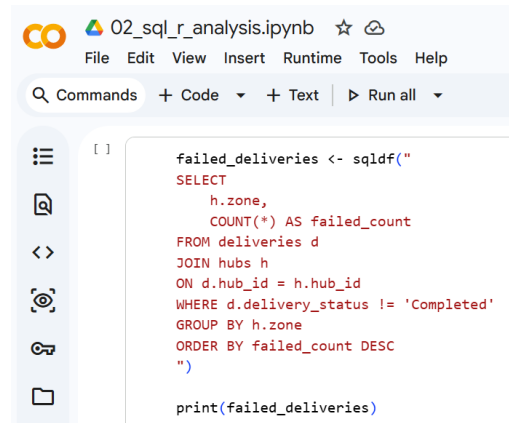
2. SQL Query Optimization Strategies

2.1 Efficient Filtering and Aggregation

SQL queries were designed to minimize unnecessary processing by:

- selecting only required columns
- applying filtering conditions early
- grouping only relevant records
- reducing redundant joins

Example Optimized Query

A screenshot of a Jupyter Notebook interface. The top bar shows the file name '02_sql_r_analysis.ipynb' and standard menu items: File, Edit, View, Insert, Runtime, Tools, Help. Below the menu is a search bar and tabs for 'Commands', '+ Code', '+ Text', and a 'Run all' button. On the left is a sidebar with icons for file explorer, search, and other notebook functions. The main area displays a code cell with the following SQL query:

```
failed_deliveries <- sqldf("
SELECT
  h.zone,
  COUNT(*) AS failed_count
FROM deliveries d
JOIN hubs h
ON d.hub_id = h.hub_id
WHERE d.delivery_status != 'Completed'
GROUP BY h.zone
ORDER BY failed_count DESC
")

print(failed_deliveries)
```

2.2. Optimization Benefits

This query improves efficiency by:

- filtering failed deliveries before aggregation
- grouping only required operational zones
- avoiding unnecessary dataset expansion

This reduces computational overhead and improves analytical performance.

2.3 Use of Indexed Identifiers

The analysis relied heavily on relational identifiers including:

- order_id
- customer_id
- delivery_id
- vehicle_id
- hub_id

These fields are suitable indexing candidates because they are:

- frequently joined
- commonly filtered
- operationally important

2.4 Business Justification

Indexing operational identifiers improves:

- delivery tracking
- complaint investigation
- operational reporting
- cross-functional analytics

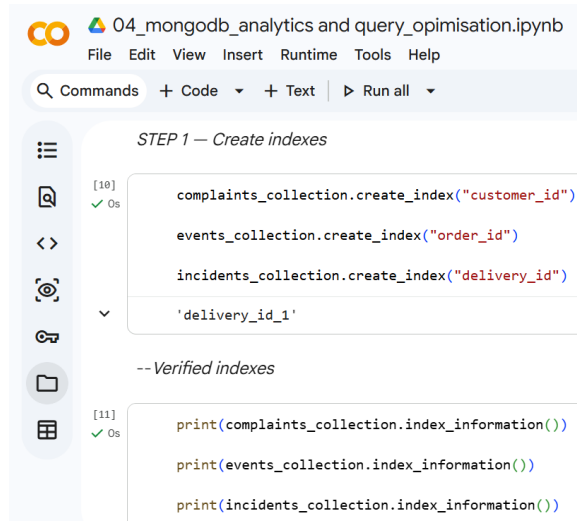
3. MongoDB Query Optimization

MongoDB optimization focused on improving retrieval speed for operational event histories and customer case tracking.

3.1 Index Creation

Indexes were created on frequently queried fields.

Complaint Collection, App Events Collection and Incident Collection Index:



The screenshot shows a Jupyter Notebook interface with the title "04_mongodb_analytics and query_opimisation.ipynb". The notebook is in "Code" view. The first code cell, labeled "[10] ✓ 0s", contains the following Python code to create indexes on three collections:

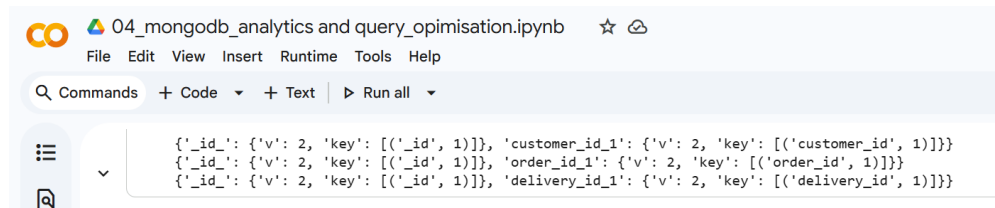
```
complaints_collection.create_index("customer_id")
events_collection.create_index("order_id")
incidents_collection.create_index("delivery_id")
```

The second code cell, labeled "[11] ✓ 0s", contains the following Python code to verify the indexes:

```
print(complaints_collection.index_information())
print(events_collection.index_information())
print(incidents_collection.index_information())
```

The notebook interface includes a sidebar with icons for file explorer, search, and other functions. The top bar shows the file name and a menu with options like File, Edit, View, Insert, Runtime, Tools, and Help.

Output



```
{ '_id': { 'v': 2, 'key': [ ( '_id', 1 ) ] }, 'customer_id_1': { 'v': 2, 'key': [ ( 'customer_id', 1 ) ] } }
{ '_id': { 'v': 2, 'key': [ ( '_id', 1 ) ] }, 'order_id_1': { 'v': 2, 'key': [ ( 'order_id', 1 ) ] } }
{ '_id': { 'v': 2, 'key': [ ( '_id', 1 ) ] }, 'delivery_id_1': { 'v': 2, 'key': [ ( 'delivery_id', 1 ) ] } }
```

3.2 Indexing Benefits

The indexing strategy improves:

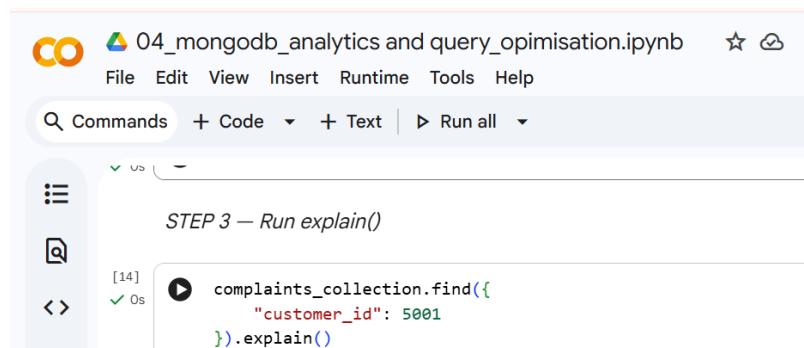
- customer complaint lookup speed
- operational event retrieval
- delivery incident investigations
- scalability for large operational datasets

Indexed retrieval reduces full collection scanning and improves analytical response time.

4. Explain Plan Analysis

MongoDB explain plans were used to evaluate query execution performance.

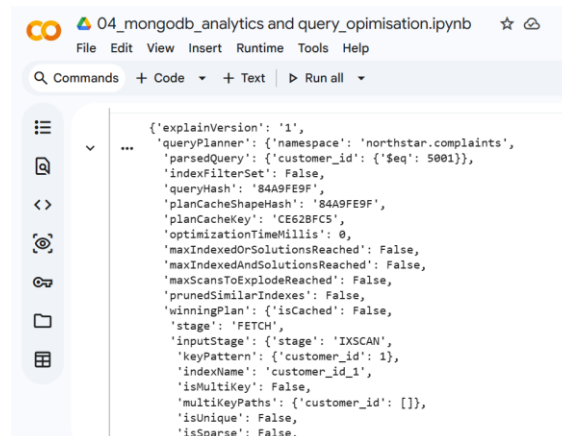
Example Query



```
STEP 3 — Run explain()

[14] ✓ 0s complaints_collection.find({
      "customer_id": 5001
    }).explain()
```

Output:

A screenshot of a Jupyter Notebook interface. The title bar shows 'O4_mongodb_analytics and query_opimisation.ipynb'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. The left sidebar contains icons for file explorer, search, and other notebook functions. The main area displays a JSON document representing a MongoDB explain plan. The document includes fields like 'explainVersion', 'queryPlanner', 'parsedQuery', 'indexFilterSet', 'queryHash', 'planCacheShapeHash', 'planCacheKey', 'optimizationTimeMillis', 'maxIndexedOrSolutionsReached', 'maxIndexedAndSolutionsReached', 'maxScansToExplodeReached', 'prunedSimilarIndexes', 'winningPlan', 'stage', 'inputStage', 'keyPattern', 'indexName', 'isMultiKey', 'multiKeyPaths', 'isUnique', and 'isSparse'.

```
{
  'explainVersion': '1',
  'queryPlanner': {
    'namespace': 'northstar.complaints',
    'parsedQuery': {
      'customer_id': { '$eq': 5001 }
    },
    'indexFilterSet': false,
    'queryHash': '84A9FE9F',
    'planCacheShapeHash': '84A9FE9F',
    'planCacheKey': 'CE628FC5',
    'optimizationTimeMillis': 0,
    'maxIndexedOrSolutionsReached': false,
    'maxIndexedAndSolutionsReached': false,
    'maxScansToExplodeReached': false,
    'prunedSimilarIndexes': false,
    'winningPlan': {
      'isCached': false,
      'stage': 'FETCH',
      'inputStage': {
        'stage': 'IXSCAN',
        'keyPattern': {
          'customer_id': 1
        },
        'indexName': 'customer_id_1',
        'isMultiKey': false,
        'multiKeyPaths': {
          'customer_id': []
        },
        'isUnique': false,
        'isSparse': false,

```

4.1 Explain Plan Interpretation

The explain plan demonstrated whether MongoDB used:

- collection scans (less efficient)
or
- indexed retrieval (more efficient)

Indexed queries significantly reduce document scanning and improve response efficiency for operational analytics.

5. Performance Comparison

Before Indexing

Without indexing:

- MongoDB performs full collection scans
- query execution becomes slower as data volume increases
- operational analytics become less scalable

After Indexing

With indexing:

- targeted document retrieval occurs
- fewer records are scanned
- query execution becomes significantly faster
- operational reporting improves

Example Performance Observation

The indexed `customer_id` field improved complaint retrieval efficiency by enabling direct lookup of customer case histories without scanning the full complaints collection.

6. Query Optimization in Analytical Workflows

Query optimization was important because NorthStar's operational environment contains:

- large transaction volumes
- interconnected datasets
- rapidly growing event histories
- frequent analytical reporting requirements

Efficient querying supports:

- faster management reporting
- improved operational visibility
- scalable analytics
- better customer service investigation

7. Operational Impact of Optimization

The optimization strategy directly supports:

- real-time operational monitoring
- rapid complaint investigation
- scalable delivery analytics
- faster incident resolution
- improved customer experience management

Without optimization, the fragmented and growing operational environment would become increasingly difficult to analyze efficiently.

8. Summary of Optimization Findings

The query optimization strategy improved analytical performance across both SQL and MongoDB environments through:

- efficient filtering
- aggregation optimization
- indexed retrieval

- reduced collection scanning
- improved query scalability

The findings demonstrate that optimization is essential for supporting integrated analytics within large-scale operational environments such as NorthStar Urban Mobility and Logistics.

Efficient querying enables faster decision-making, improved operational transparency, and more scalable analytical workflows.

9. Discussion and Recommendations

Key Findings

- high failure zones
- route overrides linked to incidents
- battery health affects delivery reliability
- complaint severity linked to failed deliveries

Recommendations

- improve route optimization systems
- predictive maintenance monitoring
- integrated complaint + delivery analytics
- zone-specific operational strategies
- unified data architecture using MongoDB

10. Conclusion

This analysis demonstrated that NorthStar’s operational inefficiencies are caused by interconnected issues involving route planning, vehicle condition, operational incidents, and fragmented customer service management systems.

The integration of SQL within R, statistical analytics, Python-based data processing, and MongoDB Atlas NoSQL modelling enabled the identification of important operational patterns and performance weaknesses across hubs and service zones.

The findings demonstrate the importance of combining structured and semi-structured analytics to support scalable operational intelligence and improved decision-making within modern mobility and logistics environments.

11. References

- MongoDB Inc. (2025) *MongoDB Atlas Documentation*. Available at: [MongoDB Atlas](#) (Accessed: 15 May 2026).
- McKinney, W. (2022) *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*. 3rd edn. Sebastopol: O'Reilly Media.
- Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. 2nd edn. Cham: Springer.
- Grothendieck, G. (2017) *sqldf: Manipulate R Data Frames Using SQL*. Available at: [CRAN sqldf Package](#) (Accessed: 15 May 2026).
- Van Rossum, G. and Drake, F.L. (2009) *Python 3 Reference Manual*. Scotts Valley: CreateSpace.
- Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95.
- The pandas development team (2025) *pandas Documentation*. Available at: [pandas Documentation](#) (Accessed: 15 May 2026).
- NumPy Developers (2025) *NumPy Documentation*. Available at: [NumPy Documentation](#) (Accessed: 15 May 2026).
- PyMongo Developers (2025) *PyMongo Documentation*. Available at: [PyMongo Documentation](#) (Accessed: 15 May 2026).
- Google (2025) *Google Colaboratory Documentation*. Available at: [Google Colab](#) (Accessed: 15 May 2026).